

Document d'Architecture Technique

Plateforme CI/CD et environnement DevOps industrialisé

Client

Korlea

Équipe projet

Amine Messadi • Karim Haddadi

Tom Patelas • Nicolas Roulois

Mastère DevOps M1, SUP DE VINCI

Promotion M1DOA 2025-2026

Table des matières

Partie 1. Présentation de l'entreprise et de l'équipe projet.....	5
1.1 Le client : une ESN et son modèle de livraison	5
1.2 Contexte d'apparition du projet	5
1.3 L'équipe projet et la répartition des rôles	6
1.4 Objet et périmètre du document.....	6
Partie 2. Analyse de la problématique et introduction à la solution proposée	7
2.1 Diagnostic.....	7
2.1.1 Énoncé des cinq enjeux	7
2.1.2 Cercle vicieux des dysfonctionnements actuels	7
2.1.3 Attentes implicites.....	7
2.2 Objectifs et critères de succès	8
2.2.1 Traduction des enjeux en critères mesurables.....	8
2.2.2 Périmètre fonctionnel cible	8
2.3 Cartographie des solutions envisagées.....	9
2.4 Vision d'ensemble de la solution retenue	10
2.4.1 Principe directeur	10
2.4.2 Schéma macro	10
2.4.3 Synthèse des briques retenues.....	10
2.5 Traçabilité des exigences vers la solution	12
2.6 Limitations consciemment assumées	14
Partie 3. Organisation de l'équipe, planification détaillée et méthodologies appliquées	15
3.1 Cadre méthodologique	15
3.1.1 Choix d'une approche Scrumban.....	15
3.1.2 Cérémonies et rythme	15
3.1.3 Definition of Done	15
3.2 Rôles et responsabilités	16
3.2.1 Quatre rôles, six domaines techniques	16
3.2.2 Matrice RACI.....	16
3.3 Outillage de gestion projet	17
3.3.1 Versioning et hébergement Git	17
3.3.2 Board Kanban	17
3.3.3 Communication	17
3.3.4 Documentation continue.....	17
3.4 Planification détaillée.....	18
3.4.1 Jalons macro	18
3.4.2 Diagramme de Gantt	18
3.4.3 Sprints et thèmes.....	18
3.5 Gestion des risques	19
3.5.1 Risques identifiés au kick-off	19
3.5.2 Mitigations effectives	19
3.5.3 Risques résiduels	20
Partie 4. Présentation de la solution technique	21
Avant-propos	21

4.1 Architecture d'ensemble.....	21
4.1.1 Vue logique.....	21
4.1.2 Vue physique	22
4.1.3 Vue flux de données	23
4.1.4 Principes directeurs	23
4.2 Couche infrastructure : NixOS sur Proxmox	24
4.2.1 Choix de Proxmox VE bare-metal	24
4.2.2 Choix de NixOS 25.11.....	24
4.2.3 Structure du flake	25
4.2.4 Partitionnement via Disko	25
4.2.5 Provisionnement par nixos-anywhere.....	26
4.2.6 Gestion des secrets via sops-nix	26
4.2.7 Exposition par Tailscale serve.....	27
4.3 Conteneurisation	28
4.3.1 Choix de l'image runtime distroless	28
4.3.2 Dockerfile multi-stage à trois cibles	28
4.3.3 Optimisations binaire et taille d'image	29
4.3.4 SecurityContext et durcissement runtime.....	29
4.3.5 Stratégie de tag et de cache GHA.....	30
4.4 Orchestration Kubernetes via K3s.....	31
4.4.1 Choix de K3s.....	31
4.4.2 Topologie et flags k3s	31
4.4.3 Cartographie des namespaces.....	32
4.4.4 Bootstrap déclaratif via k8s-bootstrap.nix	33
4.4.5 ApplicationSet GitOps multi-catégorie.....	33
4.4.6 Image Updater write-back Git et digest pinning	34
4.4.7 Ressources de l'application	34
4.4.8 Probes et stratégie de déploiement	35
4.5 CI/CD	36
4.5.1 Choix de GitHub Actions et de GHCR.....	36
4.5.2 Pipeline application	37
4.5.3 Pipeline infrastructure.....	38
4.5.4 Triggers, artefacts et secrets	39
4.5.5 Métriques CI observées	39
4.6 Application métier	40
4.6.1 Choix de Go 1.25 et de HTMX.....	40
4.6.2 Architecture cmd/internal et trois binaires.....	41
4.6.3 Stockage Postgres via CloudNative-PG.....	42
4.6.4 Worker LISTEN/NOTIFY et propagation de trace.....	43
4.6.5 CronJob audit-purge et conformité RGPD	44
4.6.6 Authentification OIDC et RBAC à trois rôles	44
4.6.7 Endpoints chaos pour démonstration	45
4.6.8 Tests d'intégration via testcontainers-go	46
4.7 Sécurité	47
4.7.1 Modèle de menace simplifié	47
4.7.2 Gestion des secrets en trois couches	47
4.7.3 RBAC Kubernetes natif et chaos-killer scoped.....	48
4.7.4 NetworkPolicies par composant applicatif	49
4.7.5 Scan d'image en deux temps.....	49
4.7.6 Terminaison TLS par Tailscale serve	50
4.7.7 Conformité RGPD	51

4.7.8 Limitations assumées et conditions de passage en production	51
4.8 Observabilité	53
4.8.1 Triade métriques, logs, traces	53
4.8.2 Stack kube-prometheus-stack, Loki, Alloy, Tempo, Kepler	54
4.8.3 Métriques applicatives et exemplars.....	54
4.8.4 Alerting SLO multi-burn-rate et webhook Discord	55
4.8.5 Dashboards Grafana	56
4.8.6 Corrélation tri-directionnelle.....	57
4.9 Résilience et autoscaling.....	58
4.9.1 Autoscaling horizontal des Pods.....	58
4.9.2 PodDisruptionBudget	59
4.9.3 Probes startup, readiness, liveness	59
4.9.4 Stratégie de déploiement et GitOps self-heal	60
4.9.5 Chaos engineering	61
4.10 Green IT.....	62
4.10.1 Mesure énergétique par Kepler	62
4.10.2 Autoscaling horizontal comme levier de sobriété.....	62
4.10.3 Images conteneurs minimales.....	63
4.10.4 Mutualisation du cache CI	63
4.10.5 Rétentions courtes des données d'observabilité	64
4.10.6 Reporting Green IT et limites connues	64
4.11 KPIs et résultats observés	65
4.11.1 KPIs ancrés au snapshot du 24 mai 2026	65
4.11.2 KPIs à actualiser au jour de soumission.....	67
4.11.3 Synthèse architecture quantifiée	68
4.12 Limites techniques et perspectives.....	68
Glossaire technique.....	70

Partie 1. Présentation de l'entreprise et de l'équipe projet

1.1 Le client : une ESN et son modèle de livraison

Korlea est une entreprise de services numériques basée à Paris. Son activité couvre la conception, le développement et l'exploitation d'applications web destinées à des PME et à des collectivités locales. Le facteur de différenciation commerciale repose sur la capacité à livrer rapidement de nouvelles versions, sans régression, et avec une qualité homogène d'un client à l'autre.

L'entreprise s'articule autour de quatre pôles aux préoccupations distinctes. Le pôle Développement attend une automatisation complète des builds, des tests et des livraisons. Le pôle Exploitation demande des déploiements rapides, un rollback simple et une scalabilité maîtrisée. Le pôle Sécurité réclame des scans automatisés de vulnérabilités, une gestion centralisée des secrets et une conformité RGPD démontrable. La direction recherche une visibilité temps réel sur les performances applicatives et sur la consommation énergétique de l'infrastructure. Cette grille de lecture, héritée du cahier des charges (*docs/cahier-des-charges.pdf* §III), structure l'ensemble des choix techniques argumentés en Partie 4.

Une nuance s'impose. Korlea ne dispose pas de RSSI dédié : la fonction sécurité reste portée par le CTO. Ce point justifie une approche security-by-design, intégrée au pipeline CI/CD et au cluster lui-même, plutôt qu'un audit a posteriori conduit par une équipe spécialisée.

1.2 Contexte d'apparition du projet

L'héritage technique reste typique d'une ESN française de cette taille. La majorité des applications repose sur PHP/Symfony. Les déploiements s'effectuent manuellement par SSH et rsync, à partir de scripts shell maison qui varient d'un projet à l'autre. Chaque client se voit attribuer une VM dédiée, configurée à la main.

Le cahier des charges (*docs/cahier-des-charges.pdf* §I) identifie cinq dysfonctionnements directs :

- les builds et déploiements manuels varient selon les équipes, ce qui produit des comportements non reproductibles ;
- les tests ne sont pas systématisés, et les régressions atteignent régulièrement la production ;
- aucun monitoring centralisé ne signale qu'un service est surchargé, instable ou anormalement énergivore ;
- les environnements restent hétérogènes (mélange de VMs, conteneurs isolés, machines physiques), ce qui complique toute scalabilité horizontale ;
- la sécurité de la chaîne logicielle reste artisanale : aucun scan automatisé, secrets stockés en clair dans des fichiers de configuration ou transmis par messagerie.

La perte récente d'un appel d'offre, faute de garanties opérationnelles formalisables sur la chaîne de livraison, déclenche la décision de la direction d'investir dans une plateforme DevOps industrielle. C'est l'objet du présent projet.

1.3 L'équipe projet et la répartition des rôles

Quatre étudiants composent l'équipe : Amine Messadi, Karim Haddadi, Tom Patelas, Nicolas Roulois. Le choix d'un groupe de quatre couvre l'intégralité des domaines techniques M1 DevOps mobilisés par le sujet. La répartition s'organise par domaine de spécialisation, sans silo étanche : chaque membre contribue à l'ensemble du projet, mais porte la responsabilité éditoriale de son périmètre lors des revues internes et de la rédaction du livrable.

Membre	Domaine	Responsabilités
Tom Patelas	Infrastructure et GitOps	Flake NixOS, scripts Proxmox, modules K3s server et agent, sops-nix, bootstrap ArgoCD, justfile
Amine Messadi	CI/CD et qualité logicielle	Workflows GitHub Actions, linters, validateurs de manifests, argocd-image-updater
Karim Haddadi	Conteneurisation et application Go	Dockerfile multi-stage distroless, code applicatif (api, worker, audit-purge), tests d'intégration testcontainers-go, CronJob de purge RGPD
Nicolas Roulois	Observabilité et sécurité	Stack kube-prometheus-stack, Loki, Alloy et Tempo, dashboards Grafana, règles d'alerting SLO, Vault, Dex OIDC, Kepler, NetworkPolicies applicatives

La matrice RACI détaillée par domaine technique figure en §3.2.2. La traçabilité des contributions reste portée par l'historique Git et les revues croisées de pull requests.

1.4 Objet et périmètre du document

Le présent document est le livrable de groupe du projet d'études. Il décrit la plateforme conçue pour répondre au cahier des charges Korlea, justifie les choix techniques retenus, formalise la solution livrée, et expose les limites consciemment assumées. Sa structure suit le plan imposé par le cadre pédagogique (*docs/cadre-pedagogique.pdf*) : présentation de l'entreprise et de l'équipe, analyse de la problématique, organisation et méthodologies, présentation technique de la solution.

Trois objets restent en dehors du périmètre de ce document :

- les rapports individuels des quatre membres, livrés séparément, qui couvrent les perspectives d'évolution, l'analyse critique des limites techniques et les annexes personnelles propres à chaque périmètre ;
- la gestion des coûts d'infrastructure (FinOps), thème consolidé en M2 ;
- la haute disponibilité multi-environnement, la duplication multi-région et le plan de reprise d'activité formalisé, qui dépassent le cadre des trois VMs Proxmox du projet d'études.

Ces trois axes sont rappelés en §2.6 sous l'angle des limitations assumées qui en découlent.

Partie 2. Analyse de la problématique et introduction à la solution proposée

2.1 Diagnostic

2.1.1 Énoncé des cinq enjeux

Le cahier des charges (*docs/cahier-des-charges.pdf* §II) énonce cinq enjeux qui structurent l'ensemble du projet :

- réduire le temps de mise en production de 30 % ;
- fiabiliser les livraisons en réduisant les incidents post-déploiement ;
- améliorer la supervision via des indicateurs métiers et techniques temps réel ;
- sécuriser la chaîne logicielle (scans, secrets, conformité) ;
- réduire l'empreinte carbone de l'infrastructure (Green IT).

Ces cinq enjeux ne sont pas indépendants. La fiabilité dépend de la supervision. La supervision conditionne la mesure d'empreinte. La sécurité contraint la chaîne CI/CD. Le temps de mise en production repose sur l'ensemble. La solution retenue répond donc aux cinq en bloc plutôt qu'individuellement.

2.1.2 Cercle vicieux des dysfonctionnements actuels

Les cinq dysfonctionnements décrits en Partie 1 (§1.2) forment un cercle vicieux. Builds manuels et hétérogénéité d'environnement produisent des comportements imprévisibles en production. L'absence de tests systématisés laisse passer des régressions, qui ne sont détectées qu'après remontée client. L'absence de monitoring centralisé prolonge à la fois le temps de détection (MTTD) et le temps de résolution (MTTR). La gestion artisanale des secrets ajoute un risque de divulgation et complique toute rotation. L'hétérogénéité applicative interdit d'industrialiser une réponse globale : chaque incident requiert une investigation spécifique.

Trois conséquences sont mesurables côté client. Le délai de mise en production d'une modification non triviale dépasse régulièrement la journée ouvrée. Les incidents post-déploiement représentent une part significative des sollicitations TMA. Aucune fenêtre de maintenance n'est planifiable sans intervention humaine sur chaque VM cible.

2.1.3 Attentes implicites

Le cahier ne formule pas explicitement quatre attentes, mais elles se déduisent du contexte client et du marché.

La conformité RGPD s'impose dès lors que Korlea adresse des collectivités locales, soumises à des obligations strictes sur les données personnelles : journalisation des accès, droit à l'effacement, traçabilité.

Une scalabilité horizontale par défaut, sans intervention manuelle, conditionne la croissance du portefeuille clients sans réorganisation de l'infrastructure à chaque montée en charge.

Une observabilité tri-canal (métriques, logs, traces) démontre la maîtrise opérationnelle sur l'ensemble du cycle de vie d'une requête, au-delà du seul monitoring CPU/RAM.

Une sobriété infrastructurelle assumée répond à la fois à l'objectif Green IT explicite et à une logique de coût d'exploitation.

2.2 Objectifs et critères de succès

2.2.1 Traduction des enjeux en critères mesurables

Chaque enjeu se traduit en un ou plusieurs critères mesurables. Le tableau formalise les cibles retenues. La cible de p95 sous 1 s à 200 RPS est calibrée sur le dimensionnement HPA et le matériel disponible, et ne constitue pas un SLA contractuel.

Enjeu	Critère mesurable	Cible projet d'études
Temps de mise en production -30 %	Deploy time bout en bout (push Git → pod ready)	sous 5 minutes
Fiabilité des livraisons	Taux de succès pipeline <code>build.yml</code>	au-dessus de 90 % sur les 30 derniers runs
Supervision temps réel	Couverture observabilité (métriques, logs, traces)	100 % des services applicatifs
Performance applicative sous charge	p95 latence HTTP soutenue à 200 RPS	sous 1 s
Sécurité de la chaîne logicielle	CVE HIGH/CRITICAL dans une image acceptée	zéro
Conformité RGPD	Audit log applicatif et purge automatique	rétention 90 jours configurée
Empreinte carbone	Conso énergétique mesurée par pod	reporting Kepler actif

Le cahier des charges (*docs/cahier-des-charges.pdf* §IX) fixe également quatre KPI minimums à suivre : temps moyen de déploiement, taux de réussite des builds, nombre de vulnérabilités détectées, consommation CPU/RAM par environnement. Ces quatre KPI sont nativement couverts par la stack retenue et reportés en Partie 4 (§4.11).

2.2.2 Périmètre fonctionnel cible

Pour la mission projet d'études, le périmètre fonctionnel retenu comporte six volets :

- une plateforme socle reproductible : infrastructure déclarative, cluster d'orchestration, GitOps, gestion de secrets centralisée ;
- une application métier représentative : un portail de tickets HTMX avec API REST, worker asynchrone et job RGPD, choisi pour démontrer toute la chaîne (rendu HTML, persistance Postgres, jobs queue, audit log) sur un cas concret ;
- une chaîne CI/CD bout en bout : lint, tests automatisés, build conteneurisé, scan de sécurité bloquant, livraison continue déclarative ;
- une couche observabilité complète : métriques, logs, traces, alerting, dashboards métier ;

- une démonstration de résilience : chaos engineering, autoscaling, alertes burn-rate ;
- un volet Green IT mesurable : suivi énergétique par pod, optimisation des images, scaling à la charge.

2.3 Cartographie des solutions envisagées

Pour chaque couche technique, plusieurs alternatives ont été instruites. La synthèse ci-dessous résume le choix retenu et les options écartées, avec un renvoi vers la justification fine en Partie 4.

Couche	Alternatives écartées	Choix retenu	Détail
CI/CD	GitLab CI (instance à maintenir ou SaaS payant), Jenkins (configuration impérative, maintenance lourde), Drone CI et Tekton (maturité et adoption plus limitées)	GitHub Actions + GHCR	§4.5.1
Orchestration	Kubernetes vanilla (empreinte mémoire et complexité opérationnelle élevées sur trois VMs de 2 vCPU et 6 GiB), Kubernetes managé GKE/EKS/AKS (coût récurrent, dépendance cloud), Nomad (capacités natives plus limitées en supervision et RBAC)	K3s	§4.4.1
Secrets	sealed-secrets (ne couvre que la couche K8s, pas le bootstrap OS), Vault seul (ne couvre pas l'OS), sops seul (ne couvre pas les secrets runtime applicatifs)	sops-nix (OS et K3s) + Vault + External-Secrets (runtime cluster)	§4.7.2
Observabilité	Datadog ou New Relic (SaaS payant, données hors cluster), ELK (empreinte RAM trop importante face à la cible 6 GiB par VM)	kube-prometheus-stack + Loki + Tempo + Alloy + Grafana	§4.8
Infrastructure as Code	Terraform + Ansible (deux outils, deux DSL, état à gérer), cloud-init seul (pas de mise à jour idempotente après installation)	NixOS déclaratif (flake) + cinq scripts shell côté Proxmox	§4.2

Cinq critères ont guidé chaque arbitrage : reproductibilité bout en bout, du bare-metal au pod ; coût d'exploitation (RAM, CPU, licence) compatible avec trois VMs Proxmox de 2 vCPU et 6 GiB ; alignement avec les compétences M1 DOA et les attentes du jury technique ; maturité et adoption des composants éprouvés en production ailleurs ; sobriété, c'est-à-dire un overhead minimal pour le service rendu. L'arbitrage final privilégie systématiquement la déclarativité et l'absence de SaaS payant.

2.4 Vision d'ensemble de la solution retenue

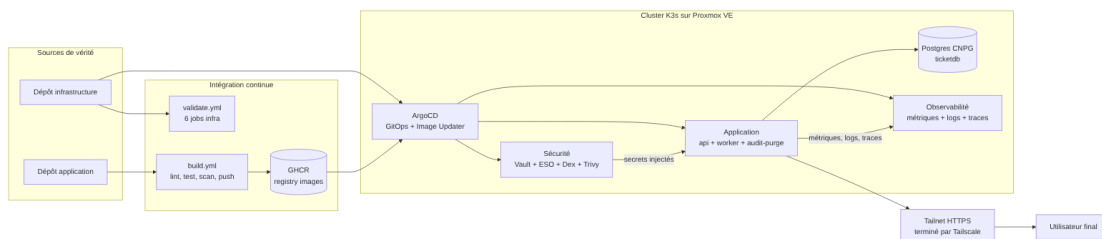
2.4.1 Principe directeur

Un seul principe directeur guide toute la solution : tout est déclaratif, tout est versionné, tout est reproductible. Le contenu des deux dépôts Git suffit, à lui seul, à reconstruire la plateforme à partir de zéro. Aucune intervention manuelle persistante n'existe en dehors des dépôts. Toute modification passe par une pull request, déclenche les pipelines CI, et se propage au cluster via la chaîne **GitOps**.

Ce principe répond aux dysfonctionnements identifiés en §2.1.2 : il supprime la dérive entre environnements, garantit la reproductibilité des builds, formalise la trace de chaque changement, et soutient la rotation des secrets sans manipulation manuelle.

2.4.2 Schéma macro

Le schéma ci-dessous donne une vue d'ensemble de la solution. Le détail de chaque sous-système figure en Partie 4 (§4.1 pour les vues complémentaires logique, physique et flux de données).



Architecture macro de la solution retenue

2.4.3 Synthèse des briques retenues

Le tableau ci-dessous résume les briques retenues par couche, avec leur version exacte. Les versions citées proviennent du snapshot du 24 mai 2026.

Couche	Composant	Version	Source
Hyperviseur	Proxmox VE	hôte personnel	machine de l'équipe
OS nœuds	NixOS	25.11	<i>nixos/flake.nix:5</i>
Cluster	K3s	suit	<i>nixos/modules/k3s-server.nix, nixos/modules/k3s-agent.nix</i>
GitOps	ArgoCD	v2.13.0	<i>nixos/flake.nix:10-13</i>
Image Updater	argocd-image-updater	0.12.0	<i>kubernetes/applications/platform/argocd-image-updater.yaml:13</i>
Secrets OS	sops-nix	rev pinné dans <i>flake.lock</i>	<i>nixos/flake.lock</i>
Secrets	Vault	0.31.0	<i>kubernetes/applications/security/vault.yaml:13</i>

runtime		(mode DEV)	
Sync secrets	External-Secrets	0.20.4	<i>kubernetes/applications/security/external-secrets.yaml:13</i>
IdP	Dex	0.20.0	<i>kubernetes/applications/security/dex.yaml:13</i>
Scan images cluster	trivy-operator	0.30.0	<i>kubernetes/applications/security/trivy-operator.yaml:13</i>
Métriques	kube-prometheus-stack	65.5.1	<i>kubernetes/applications/monitoring/kube-prometheus-stack.yaml:13</i>
Logs	Loki	6.16.0 SingleBinary	<i>kubernetes/applications/monitoring/loki.yaml:13</i>
Collecteur logs	Grafana Alloy	0.10.1 DaemonSet	<i>kubernetes/applications/monitoring/alloy.yaml:13</i>
Traces	Tempo	1.24.4 monolithic	<i>kubernetes/applications/monitoring/tempo.yaml:13</i>
Énergie	Kepler	0.5.12 (image release-0.7.12)	<i>kubernetes/applications/observability/kepler.yaml:13,21</i>
Opérateur Postgres	CloudNative-PG	0.22.1	<i>kubernetes/applications/data/cloudnative-pg.yaml:13</i>
Base de données	Postgres (CNPG)	16.4	<i>kubernetes/apps/projet-etude-app-demo/database.yaml:7</i>
Réseau VPN	Tailscale + tailscale serve	déclaratif	<i>nixos/modules/tailscale.nix, nixos/modules/tailscale-serve.nix</i>
Ingress	Traefik (intégré K3s)	défaut K3s	<i>kubernetes/applications/platform/traefik-web-nodeport.yaml</i>
Application	Go	1.25.0	<i>projet-etude-app-demo/go.mod:3</i>
CI app	GitHub Actions + GHCR	matrix 3 composants	<i>.github/workflows/build.yml</i>
CI infra	GitHub Actions	6 jobs validation	<i>.github/workflows/validate.yml</i>
Runner local	just	devShell Nix	<i>justfile</i>

2.5 Traçabilité des exigences vers la solution

La matrice ci-dessous trace les quinze exigences fonctionnelles du cahier (*docs/cahier-des-charges.pdf* §IV), plus le bonus chaos engineering, vers le composant qui les couvre. Le statut suit la convention : réalisé, partiel, hors périmètre. Les exigences partiellement couvertes sont reprises en §2.6 et détaillées en Partie 4.

#	Exigence (cahier §IV)	Composant retenu	Statut	Source
1	CI/CD complète : build, tests, scan, déploiement	GitHub Actions + Trivy + ArgoCD Image Updater	réalisé	<i>.github/workflows/build.yml, kubernetes/applications/platform/apps-applicationset.yaml</i>
2	Gestion des versions (branches, releases, rollback)	Branches Git + rollback par revert de digest	partiel (pas de tag SemVer)	historique Git
3	Conteneurisation Docker	Dockerfile multi-stage distroless	réalisé	<i>projet-etude-app-demo/Dockerfile</i>
4	Orchestration Kubernetes / K3s	K3s 1 control-plane + 2 workers	réalisé	<i>nixos/modules/k3s-server.nix, nixos/modules/k3s-agent.nix</i>
5	Collecte logs et métriques	kube-prometheus-stack + Loki + Alloy	réalisé	<i>kubernetes/applications/monitoring/</i>
6	Tableaux de bord temps réel	4 dashboards Grafana auto-importés	réalisé	<i>kubernetes/applications/platform/dashboard-overview.yaml, kubernetes/apps/projet-etude-app-demo/dashboard*.yaml</i>
7	Alerting en cas de problème	Alertmanager + 7 règles + webhook Discord	réalisé	<i>kubernetes/apps/projet-etude-app-demo/prometheusrule.yaml, nixos/modules/secrets.nix:64-79</i>
8	Scans automatiques de vulnérabilités	Trivy CI bloquant + trivy-operator	partiel	<i>.github/workflows/build.yml, kubernetes/applications/security/trivy-operator.yaml</i>

	(images, dépendances)	(Dependabot absent)		
9	RBAC	RBAC K3s natif + Role chaos-killer + 3 rôles applicatifs OIDC	réalisé	<i>kubernetes/apps/projet-etude-app-demo/chaos.yaml, internal/auth/auth.go</i>
10	Gestion sécurisée des secrets (Vault)	sops-nix + Vault DEV + External-Secrets	réalisé (Vault DEV, cf. §2.6)	<i>kubernetes/applications/security/vault.yaml, kubernetes/applications/security/external-secrets.yaml, nixos/modules/secrets.nix</i>
11	Auto-scaling en fonction de la charge	HPA api 2-8 sur CPU 60 %, worker 1-8 sur CPU 70 %	réalisé	<i>kubernetes/apps/projet-etude-app-demo/hpa.yaml, kubernetes/apps/projet-etude-app-demo/hpa-worker.yaml</i>
12	Analyse de la consommation CPU/RAM	Prometheus + cAdvisor + Kepler par pod	réalisé	<i>kubernetes/applications/monitoring/kube-prometheus-stack.yaml, kubernetes/applications/observability/kepler.yaml</i>
13	Optimisation des pipelines (Green IT)	Cache GHA scoped par composant + images distroless 25-35 MB	réalisé	<i>.github/workflows/build.yml:92-93, projet-etude-app-demo/Dockerfile</i>
14	Documentation : architecture technique	Présent DAT + schémas Mermaid + README projet	réalisé	présent document, <i>README.md</i>
15	Documentation : guide utilisateur et d'administration	README + justfile auto-documenté ; guide utilisateur final et OpenAPI absents	partiel	<i>README.md, justfile</i>
Bonus	Chaos engineering / démonstration	Endpoints code + CronJob	réalisé	<i>internal/httpapi/chaos.go, kubernetes/apps/projet-etude-app-demo/chaos.yaml, justfile:225-265</i>

de résilience chaos-killer
+ recettes
just
chaos-* +
probe dédié

2.6 Limitations consciemment assumées

Les aspects relevant du M2 (FinOps, multi-environnement, multi-région, plan de reprise d'activité formalisé) sont déjà cadrés en §1.4 comme hors périmètre du présent document. La présente section se concentre sur les limitations consciemment assumées à l'intérieur du périmètre M1, qui forment le cœur des conditions d'un passage en production et nourrissent les perspectives d'évolution traitées dans les rapports individuels.

Six limitations sont à ce jour assumées :

- Le serveur Vault tourne en mode dev avec un root token statique en clair dans le manifest (*kubernetes/applications/security/vault.yaml:25*). Ce mode évite la cérémonie d'unseal manuelle ; External-Secrets s'y connecte sans préparation. En production, le mode standalone avec stockage persistant, unseal automatisé (KMS cloud ou HSM) et politiques fines remplace ce mode DEV.
- Les variables OIDC sont commentées dans le manifest de déploiement de l'API (*kubernetes/apps/projet-etude-app-demo/deployment-api.yaml:83-104*). L'application traite toute requête comme operator, ce qui simplifie la démo. En production, ces variables sont restaurées et pointent vers Dex, déjà déployé et configurable.
- La base ticketdb tourne sur une instance unique CloudNative-PG (*kubernetes/apps/projet-etude-app-demo/database.yaml:9*). En production, un cluster à trois instances avec réplication synchrone et probes de basculement automatique est requis.
- Aucun manifest VolumeSnapshot, Velero ou backup natif CNPG n'est configuré. En production, un backup logique quotidien et un test de restauration formalisé sont nécessaires.
- Le stockage K3s par défaut est local-path, local à chaque nœud, ce qui interdit le déplacement d'un pod stateful d'un nœud à un autre. En production, un provisioner CSI distribué (Longhorn, Rook-Ceph) ou un stockage externe prend le relais.
- Les endpoints chaos (*/api/slow*, */api/flaky*, */api/panic*, */api/crash*, */api/memleak*) sont activés en permanence via `DEMO_CHAOS_ENABLED=1` dans le manifest de déploiement (*kubernetes/apps/projet-etude-app-demo/deployment-api.yaml:62-63*) pour illustrer la résilience pendant la démo. Hors démo, cette variable est retirée du Deployment.

Concernant le webhook d'alerting Discord (*nixos/modules/secrets.nix:64-79*), le service traite et stocke des données hors UE. Pour un déploiement Korlea réel, un canal interne UE-souverain (Mattermost self-hosted, Microsoft Teams en région européenne) remplace ce webhook.

Le tailnet et l'URL exacte du dépôt Git sont anonymisés dans le présent document (placeholder `<tailnet>.ts.net`, URL `git@github.com:<org>/projet-etude-M1.git`) ; les valeurs réelles ne sont connues que des membres de l'équipe.

Partie 3. Organisation de l'équipe, planification détaillée et méthodologies appliquées

3.1 Cadre méthodologique

3.1.1 Choix d'une approche Scrumban

L'équipe adopte une approche **Scrumban**, hybride entre Scrum et Kanban. Trois facteurs justifient ce choix.

La taille de l'équipe (quatre membres) rend Scrum strict trop lourd : les cérémonies dimensionnées pour cinq à neuf personnes y deviennent surdimensionnées. Le flux de travail reste essentiellement continu, avec peu de variabilité de charge entre semaines, ce qui rendrait un Kanban pur acceptable. Mais le projet impose deux jalons fixes (vidéo MVP, livrable écrit) à six mois du kick-off, ce qui exige une logique d'itération bornée, plus proche de Scrum.

Le compromis retenu :

- sprints d'une semaine, alignés sur le calendrier scolaire ;
- board Kanban sur GitHub Projects, colonnes Backlog, In Progress, Review, Done ;
- chaque membre suit la règle d'au plus deux tickets actifs en parallèle.

3.1.2 Cérémonies et rythme

Quatre cérémonies rythment l'équipe :

- daily asynchrone sur Discord, chaque matin : chaque membre poste hier, aujourd'hui, bloqueurs en quelques lignes. Pas de visio quotidienne, incompatible avec les emplois du temps en alternance ;
- planning hebdomadaire en visio, début de sprint : revue du backlog, sélection des tickets de la semaine, estimation rapide ;
- démo de fin de sprint, en interne, dernière demi-heure de la semaine : revue des livrables, tour d'écran ;
- rétrospective bimensuelle (un sprint sur deux) : ce qui a marché, ce qui n'a pas marché, action items pour les deux sprints suivants.

3.1.3 Definition of Done

Pour qu'un ticket passe en colonne Done, trois conditions doivent être réunies :

- la fonctionnalité est implémentée et testée localement par le développeur ;
- la pull request associée est approuvée par au moins un pair, et la CI est verte ;
- la modification est mergée sur main et visible dans le cluster pour tout changement applicatif ou de configuration.

Les correctifs critiques (sécurité, panne en démo) suivent une voie rapide : commit direct sur main par le contributeur principal du domaine concerné, avec PR rétrospective documentant le correctif. Cette voie reste exceptionnelle.

3.2 Rôles et responsabilités

3.2.1 Quatre rôles, six domaines techniques

La répartition par rôle a été fixée en Partie 1 (§1.3). Chaque membre porte la responsabilité éditoriale de son périmètre. Chaque domaine technique du projet se rattache à un responsable unique, ce qui évite la dilution des décisions et la double maintenance.

3.2.2 Matrice RACI

La matrice RACI ci-dessous formalise, pour chaque domaine technique, le contributeur Responsable et garant final (R/A), Consulté (C, donne un avis avant décision), Informé (I, lit le résultat).

Domaine	Tom	Amine	Karim	Nicolas
Flake NixOS et OS des nœuds	R/A	C	I	C
Bootstrap K3s et ArgoCD	R/A	C	I	C
Secrets sops-nix	R/A	C	I	C
Workflows GitHub Actions	C	R/A	C	C
Linters et validateurs CI	I	R/A	C	C
Image Updater ArgoCD	C	R/A	I	I
Dockerfile et builds	I	C	R/A	I
Application Go (cmd, internal)	I	C	R/A	I
Tests d'intégration	I	C	R/A	I
Manifests application K8s	C	C	R/A	C
Stack Prometheus, Loki, Tempo	I	I	I	R/A
Dashboards Grafana	I	I	I	R/A
Règles d'alerting et SLO	I	I	C	R/A
Vault, External-Secrets, Dex	C	C	C	R/A
NetworkPolicies et RBAC	C	C	C	R/A
Kepler et reporting Green IT	I	I	I	R/A
Documentation et DAT	C	C	C	C

La dernière ligne fait apparaître quatre C : la rédaction du DAT est conduite collectivement, chaque membre validant le périmètre qu'il connaît le mieux. Aucune section n'est la propriété d'un seul auteur.

3.3 Outillage de gestion projet

3.3.1 Versioning et hébergement Git

Deux dépôts Git séparés structurent le code source. Un dépôt couvre l'infrastructure (<org>/projet-etude-M1), un autre couvre l'application (<org>/projet-etude-app-demo). Le découplage offre des cycles de vie indépendants : un changement de chart Helm n'oblige pas à rebuild l'image Go, et vice versa.

Le modèle de branches reste simple. La branche main sert de référence. Les fonctionnalités arrivent sur des branches courtes nommées feat/..., fix/..., chore/.... Aucune branche develop ou release n'est utilisée : le tag implicite est le commit de main.

Les pull requests sont obligatoires. L'auto-merge est désactivé. Chaque PR doit recevoir au moins une approbation pair avant merge. Les CI (validate.yml côté infra, build.yml côté application) sont déclarées comme *required status checks* dans la configuration GitHub : un check rouge bloque le merge.

3.3.2 Board Kanban

GitHub Projects sert de board unique pour les deux dépôts. Quatre colonnes : Backlog, In Progress, Review (PR ouverte), Done. Chaque carte référence directement une Issue ou une PR sur l'un des deux dépôts, ce qui donne un point d'entrée unique pour suivre l'avancement du projet sans ouvrir trois interfaces différentes.

3.3.3 Communication

Discord sert de canal synchrone et asynchrone. Trois salons sont utilisés :

- #general : annonces, discussions transverses, partage de liens ;
- #daily : daily asynchrone, un thread par jour, archivage hebdomadaire ;
- #alerts : webhook Alertmanager du cluster (cf. Partie 4 §4.8.4), qui pousse les alertes burn-rate, erreurs HTTP, backlog jobs.

La présence du salon #alerts dans le même outil que les communications d'équipe rend visibles, par défaut, les incidents en démo : un membre qui consulte Discord pendant un cours voit immédiatement une dérive. Aucun bot intermédiaire n'est nécessaire.

3.3.4 Documentation continue

La documentation technique vit dans les dépôts eux-mêmes :

- *README.md* à la racine du dépôt infra : démarrage rapide, arborescence, conventions de sécurité des secrets ;
- *secrets/README.md* : gestion des secrets dev-side, rotation, recovery ;
- justfile auto-documenté : la commande `just` (sans argument) liste les 47 recettes avec une description en commentaire au-dessus de chacune ;
- annotations `runbook_url` et `dashboard_url` sur les PrometheusRule les plus critiques (cf. `kubernetes/apps/projet-etude-app-demo/prometheusrule.yaml`).

Le présent DAT consolide l'ensemble pour le jury et sert de base aux quatre rapports individuels. Aucun outil de documentation externe (Notion, Confluence) n'est utilisé : la documentation suit la même chaîne GitOps que le code et les manifests.

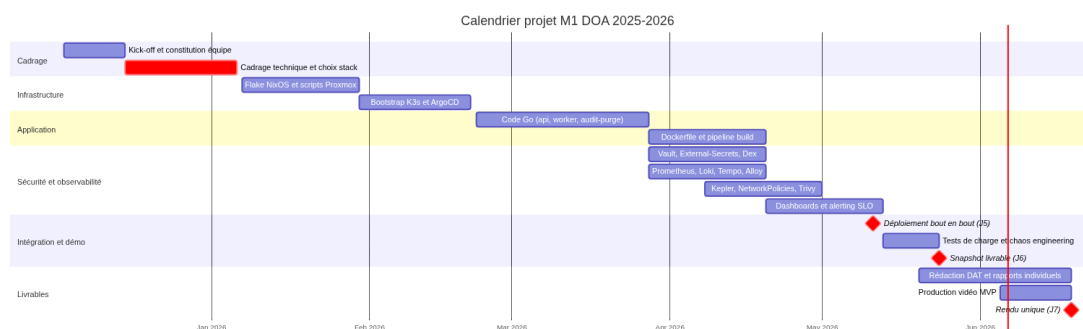
3.4 Planification détaillée

3.4.1 Jalons macro

Le projet s'articule autour de sept jalons.

- J1 kick-off, début décembre 2025 : constitution de l'équipe, choix du sujet, premier itinéraire technique.
- J2 bootstrap infrastructure, janvier à février 2026 : NixOS et scripts Proxmox opérationnels, K3s installé sur les trois VMs en environnement local de test.
- J3 pipeline applicatif, fin février à avril 2026 : code Go (api, worker, audit-purge), Dockerfile multi-stage distroless, pipeline CI build avec push GHCR.
- J4 observabilité et sécurité, fin mars à mi-mai 2026 : kube-prometheus-stack, Loki, Tempo, Alloy, Vault, External-Secrets, Dex, Kepler, NetworkPolicies, règles d'alerting SLO.
- J5 déploiement complet bout en bout : 11 mai 2026, premier `just redeploy` réussi sur les trois VMs avec l'ensemble de la stack synchronisée par ArgoCD.
- J6 snapshot livrable : 24 mai 2026, commit `afe314f` sur la branche `main`, base factuelle du présent DAT.
- J7 rendu unique : 19 juin 2026, dépôt Git de rendu contenant le présent DAT, les quatre rapports individuels et la vidéo MVP, figé au commit de rendu.

3.4.2 Diagramme de Gantt



Calendrier projet M1 DOA 2025-2026

3.4.3 Sprints et thèmes

Au total, environ 28 sprints d'une semaine séparent le kick-off du livrable final (du 3 décembre 2025 au 19 juin 2026). Quatre grandes phases thématiques émergent.

Phase 1 (sprints 1 à 6) : cadrage, choix techniques, premier flake NixOS, premier déploiement K3s local. Démo cible interne : un cluster K3s qui démarre, un pod nginx qui répond.

Phase 2 (sprints 7 à 14) : application Go, conteneurisation, pipeline CI, premier push GHCR, premier déploiement applicatif via ArgoCD. Démo cible interne : push Git, application déployée sans intervention manuelle.

Phase 3 (sprints 15 à 21) : observabilité, sécurité, Green IT, NetworkPolicies, SLO, dashboards. Démo cible interne : un dashboard Grafana qui affiche une métrique métier, une alerte qui se déclenche.

Phase 4 (sprints 22 à 28) : finalisation, chaos engineering, tests de charge, rédaction du DAT et des rapports individuels, production de la vidéo MVP. Démo cible externe : tout le système, bout en bout, à destination du client.

3.5 Gestion des risques

3.5.1 Risques identifiés au kick-off

Sept risques principaux ont été identifiés au kick-off. La matrice ci-dessous formalise leur sévérité initiale, leur probabilité initiale et la mitigation principale retenue.

Risque	Sévérité	Probabilité	Mitigation principale
Surdimensionnement technique de la solution	fort	fort	périmètre projet d'études assumé, sujets M2 listés explicitement (§1.4)
Divergence entre les 4 contributeurs sur conventions	moyen	fort	CI bloquante (lint, format, validation), revue de PR obligatoire
Instabilité d'un composant critique en démo (Vault, ArgoCD)	fort	moyen	mode DEV documenté, chaos engineering pour entraînement
Perte de temps sur l'infrastructure au détriment de l'application	moyen	fort	découpage temporel strict par phases, démos intermédiaires
Non-conformité RGPD	fort	faible	audit log et purge automatique conçus dès la phase 3
Dépendance à un service tiers (GitHub, Discord)	moyen	faible	manifests portables, GHCR remplaçable, secrets séparés
Indisponibilité matérielle (machine perso Proxmox)	fort	faible	sauvegardes du flake, possibilité de redéploiement sur VM alternative

3.5.2 Mitigations effectives

Trois mitigations méritent d'être détaillées car elles ont structuré des choix techniques de fond.

CI bloquante. Tout commit doit passer six jobs côté infra (yamllint, actionlint, shellcheck, kubeconform, nix flake check, kube-score) et trois jobs côté application (golangci-lint, test race, build avec Trivy bloquant). Un build qui échoue ne peut pas atteindre main. Le coût opérationnel reste maîtrisé (environ 35 secondes côté infra, 1 à 5 minutes côté app), et la dérive entre conventions est de facto empêchée.

Mode déclaratif total. Aucune commande manuelle ne s'exécute sur un nœud du cluster en dehors du dépôt. Le système devient reproductible : la réaction à une panne passe par `just redeploy`

plutôt que par diagnostic ad hoc. ArgoCD avec `prune: true` et `selfHeal: true` corrige toute dérive en continu (cf. *kubernetes/applications/platform/apps-applicationset.yaml:43-45*).

Séparation des secrets dev-side et cluster-side. Les secrets dev-side (clés SSH de déploiement, pre-auth Tailscale) restent sur le poste développeur, gitignored. Les secrets cluster (token K3s, webhook Discord, clé SSH image-updater) sont encryptés via sops plus age, versionnés, et décryptés au boot par chaque nœud via sa SSH host key dérivée en clé age. La trust chain reste ainsi minimale : aucun mot de passe n'est saisi manuellement à l'installation.

3.5.3 Risques résiduels

Six risques résiduels sont assumés au moment du livrable. Ils correspondent aux limitations détaillées en Partie 2 (§2.6) et seront repris en perspectives d'évolution dans les rapports individuels :

- Vault DEV en mémoire volatile, qui perd ses secrets au redémarrage du pod ;
- Postgres `ticketdb` single-instance, qui interrompt le service en cas de panne du nœud hôte ;
- absence de backup BDD formalisé ;
- stockage K3s `local-path` non répliqué entre nœuds ;
- contributeur humain unique visible dans l'historique Git, faute de configuration `Co-authored-by` systématique ;
- absence de tag SemVer Git pour formaliser les releases applicatives.

Ces risques sont tracés et ne dégradent pas la qualité de la démonstration en projet d'études.

Partie 4. Présentation de la solution technique

Avant-propos

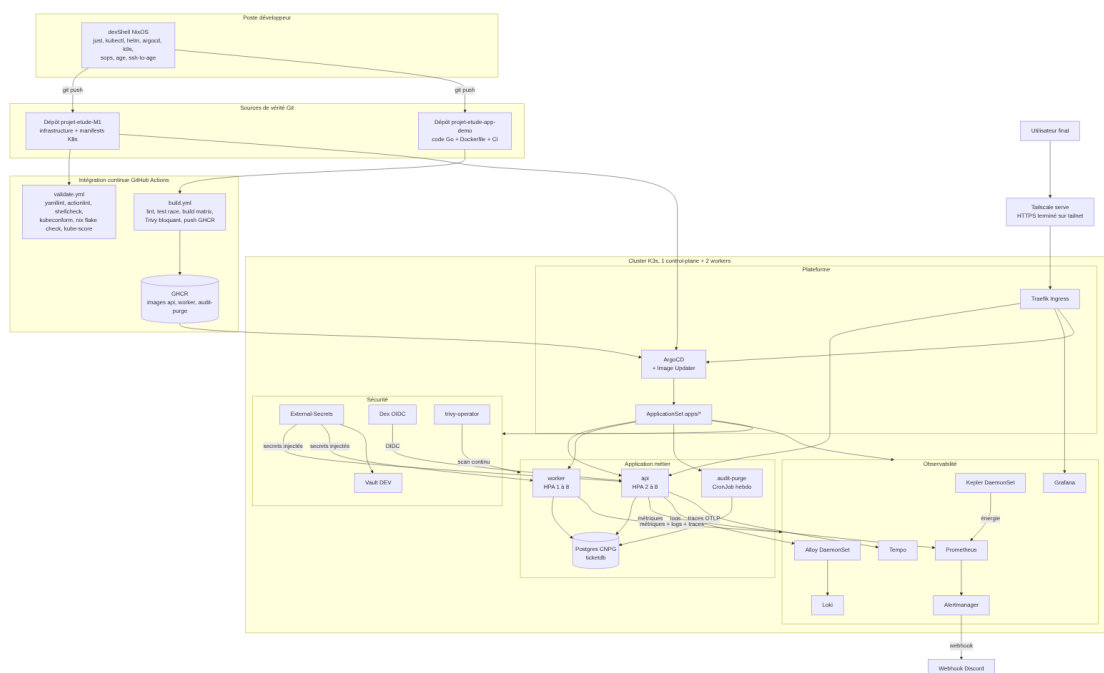
La présente partie détaille la solution technique livrée. Elle couvre l'architecture d'ensemble, chaque couche fonctionnelle (infrastructure, conteneurisation, orchestration, CI/CD, application, sécurité, observabilité, résilience, Green IT), puis les KPIs mesurés et les limites identifiées.

La gestion des coûts d'infrastructure (FinOps) ne figure pas dans cette partie. Le programme d'évaluation M1 ne l'intègre pas, et le sujet relève d'une consolidation prévue en M2 (cf. §1.4). Le présent document mentionne néanmoins la consommation CPU, RAM et énergétique mesurée, qui constitue la base technique d'une éventuelle analyse économique ultérieure.

4.1 Architecture d'ensemble

4.1.1 Vue logique

La vue logique présente les composants de la solution et leurs interactions fonctionnelles, indépendamment de leur emplacement physique. Trois sous-systèmes émergent : l'ingénierie continue (sources Git, pipelines CI, registry d'images), le cluster K3s (orchestration, application, observabilité, sécurité), et l'exposition utilisateur (Tailscale et utilisateurs finaux).



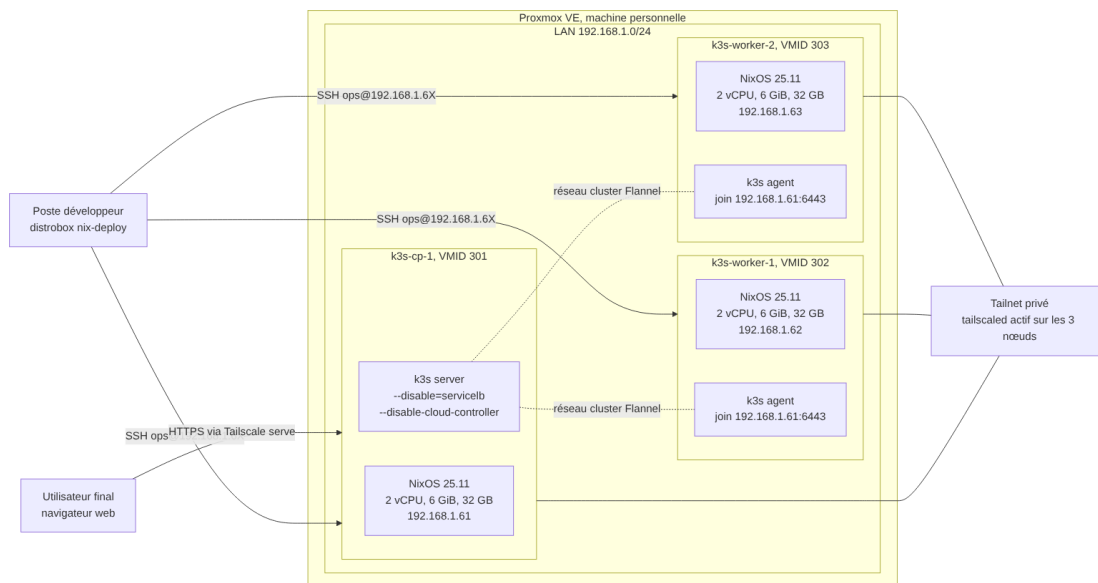
Vue logique de l'architecture

Trois principes structurants sont rendus visibles par cette vue. Le code et les manifests forment la seule source de vérité, hors du cluster ; aucune ressource n'est créée à la main. L'écriture des manifests est précédée d'une CI exhaustive qui bloque toute régression de format ou de sécurité. Le cluster est lui-même configuré par GitOps : la suppression manuelle d'une ressource est rétablie par

ArgoCD sous une minute (prune: true plus selfHeal: true, cf. `kubernetes/applications/platform/apps-applicationset.yaml:43-45`).

4.1.2 Vue physique

La vue physique présente l’emplacement des composants sur les ressources matérielles et la topologie réseau. Trois VMs Proxmox hébergent le cluster K3s. Le LAN privé 192.168.1.0/24 relie les trois nœuds. Le tailnet Tailscale relie l’utilisateur final au control-plane sans VPN traditionnel ni IP publique.

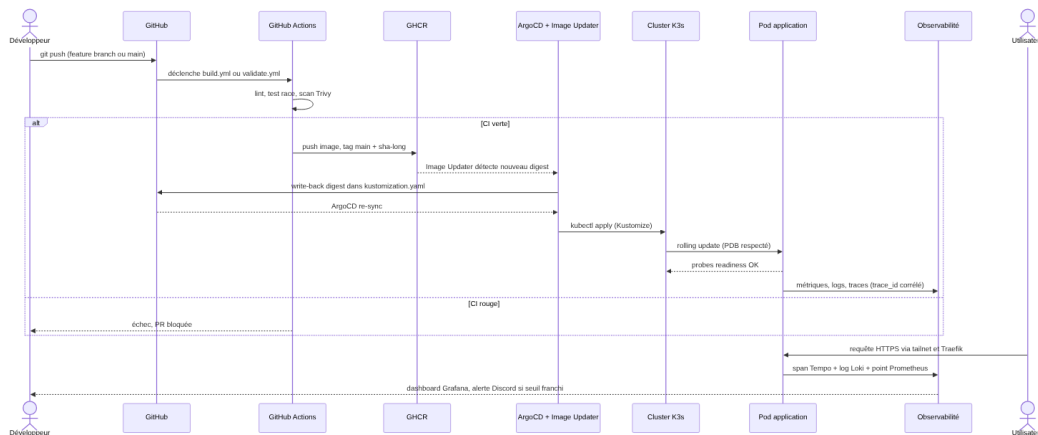


Vue physique : trois VMs Proxmox et tailnet Tailscale

Trois caractéristiques structurent cette topologie. Le cluster est volontairement dimensionné au minimum opérationnel (1 control-plane + 2 workers), ce qui correspond au seuil de tolérance à la perte d’un worker tout en gardant un quorum simple. La RAM par VM (6 GiB) reste contrainte par la machine hôte ; la stack a été choisie pour rester sous ce plafond avec une marge confortable (cf. §4.10.2). Le tailnet remplace toute exposition publique : aucun port n’est ouvert sur Internet, l’accès passe exclusivement par les credentials Tailscale, et la terminaison TLS est gérée par Tailscale serve avec un certificat Let’s Encrypt automatique.

4.1.3 Vue flux de données

La vue flux de données suit le parcours d'un changement, depuis le poste développeur jusqu'à un pod prêt à servir des requêtes, en intégrant la boucle d'observabilité.



Flux de données du push Git au Pod prêt à servir

Trois points de contrôle se distinguent dans cette séquence. La CI bloque toute image dont Trivy détecte une CVE HIGH ou CRITICAL avant push GHCR. L'Image Updater écrit le digest exact (et non un tag mutable) dans le dépôt Git, ce qui garantit la reproductibilité et le rollback par revert. ArgoCD avec `prune: true` et `selfHeal: true` rétablit toute dérive en moins d'une minute, sans intervention humaine.

4.1.4 Principes directeurs

Quatre principes directeurs sous-tendent l'ensemble de la solution. Chacun protège la plateforme contre un type de défaillance ou de dérive spécifique.

Reproductibilité bout en bout. Le contenu des deux dépôts Git, ajouté à un poste développeur muni du devShell NixOS, suffit à reconstruire la plateforme à partir de zéro. NixOS pinne l'OS, `flake.lock` pinne chaque dépendance Nix, ArgoCD est pinné à v2.13.0, les charts Helm sont pinnés version par version (cf. matrice §2.4.3), et les images applicatives sont pinnées par digest SHA. Aucune commande manuelle ne s'exécute hors des dépôts.

Sécurité by design. Le cluster ne s'expose jamais sur Internet directement : Tailscale serve termine TLS sur le tailnet. Les conteneurs tournent en `distroless`, `nonroot`, capacités ALL droppées, `readOnlyRootFilesystem` activé, profil `seccomp RuntimeDefault`. Les secrets sont chiffrés à la source et n'apparaissent jamais en clair dans le dépôt. La CI bloque toute CVE HIGH ou CRITICAL avant push. Le cluster scanne en continu les images déployées via `trivy-operator`.

Observabilité tri-canal. Chaque requête HTTP génère un span Tempo, un point Prometheus et un log Loki, tous corrélés par `trace_id`. Les exemplars Prometheus pointent depuis un bucket d'histogramme vers la trace Tempo correspondante. Les derived fields Loki pointent depuis un log vers Tempo et Prometheus. Le triangle métriques-logs-traces se navigue dans Grafana sans changer d'outil.

Sobriété par défaut. La stack est dimensionnée au minimum opérationnel. Les images applicatives pèsent entre 25 et 35 MB. Le cache CI mutualise les couches Docker scope par composant. Kepler mesure et publie la consommation énergétique de chaque pod. L'auto-scaling ramène le nombre de

répliques au minimum quand la charge baisse. Les rétentions Loki et Tempo sont fixées à sept jours, suffisantes pour les démos et l'investigation post-incident, et compatibles avec les contraintes RGPD.

4.2 Couche infrastructure : NixOS sur Proxmox

4.2.1 Choix de Proxmox VE bare-metal

L'infrastructure repose sur Proxmox VE installé sur une machine personnelle. Trois critères ont écarté les alternatives :

- VMware ESXi : modèle de licence incompatible avec un budget projet d'études nul. La version gratuite vSphere Hypervisor a été dépréciée en février 2024, remplacée par des éditions payantes uniquement.
- Cloud public (AWS, GCP, OVH) : coût récurrent et dépendance externe, écartés pour les mêmes raisons que dans le comparatif §2.3.
- KVM nu sans Proxmox : possible techniquement, mais oblige à écrire l'automatisation autour (provisionnement, snapshots, console série) que Proxmox apporte clé en main.

Proxmox VE coche les trois exigences : pas de licence, API qm accessible par SSH, support des templates cloud-init et qcow2 standards. Le template Debian 12 genericcloud est récupéré depuis cloud.debian.org par `proxmox/create-template.sh`, ce qui supprime toute construction d'image custom (VMID 9000 réservé au template).

Les trois VMs cibles sont créées par `proxmox/create-vm.sh:17-29` à partir du template, avec un dimensionnement identique : VMID 301 à 303, 2 vCPU, 6 144 MiB de RAM, 32 GB de disque. Le dimensionnement reste volontairement minimal, contraint par la machine hôte, et cohérent avec K3s léger (cf. §4.4.1).

4.2.2 Choix de NixOS 25.11

NixOS 25.11 (stable, pinné dans `nixos/flake.nix:5`) sert d'OS aux trois nœuds. Plusieurs alternatives ont été instruites :

- Debian 12 et Ubuntu 24.04 LTS : stables, large catalogue de paquets, mais configuration impérative via Ansible ou scripts shell, état entre installation et exploitation difficile à figer, rollback transactionnel inexistant.
- AlmaLinux 9 : équivalent côté RHEL, mêmes limites.
- Talos Linux : OS minimaliste 100 % orienté Kubernetes, déclaratif, mais réservé strictement à un cluster K8s. Aucun shell, aucune flexibilité pour les besoins hors K8s (Tailscale en service système, justfile, scripts de debug).

NixOS répond à quatre exigences spécifiques du projet : configuration déclarative bout en bout via `flake.nix` (OS, packages, services, secrets, exposition tailnet) ; pinning précis via `flake.lock`, chaque dépendance étant ancrée à un commit Git précis (y compris ArgoCD v2.13.0, sops-nix, disk) ; rollback atomique natif via `nixos-rebuild switch --rollback` ; DevShell embarqué reproductible accessible par `nix develop ./nixos` (cf. `nixos/flake.nix:72-93`).

Le coût d'entrée (apprentissage du langage Nix) est assumé. L'équipe gagne en contrepartie une infrastructure dont le redéploiement complet (`just redeploy`) reproduit exactement l'état d'origine, sans dérive cumulée.

4.2.3 Structure du flake

Le **flake** (*nixos/flake.nix*, 95 lignes) déclare quatre inputs : `nixpkgs` sur la branche `nixos-25.11`, `disko`, `argo-cd` épinglé à `v2.13.0` (`flake = false`, car les manifests ArgoCD sont consommés directement), et `sops-nix`. Les révisions exactes figurent dans *nixos/flake.lock*.

Le cœur du flake est la fonction `mkNode` (*nixos/flake.nix:25-43*), qui assemble pour un hôte donné :

- les modules transverses : *disko-proxmox.nix*, *common.nix*, *secrets.nix*, *tailscale.nix*, *tailscale-serve.nix* ;
- le module spécifique au rôle : *k3s-server.nix* ou *k3s-agent.nix* ;
- le fichier hôte : *hosts/k3s-cp-1.nix*, *hosts/k3s-worker-1.nix*, *hosts/k3s-worker-2.nix* ;
- conditionnellement, *modules/k8s-bootstrap.nix*, qui n'est ajouté que sur les nœuds server pour poser ArgoCD et la root app au boot.

Trois entrées *nixosConfigurations* sont exposées : `k3s-cp-1`, `k3s-worker-1`, `k3s-worker-2` (*nixos/flake.nix:62-66*). Un package supplémentaire `projet-kubeconfig` (*nixos/flake.nix:46-60*) récupère le kubeconfig depuis le control-plane via SSH et l'écrit dans `~/ .kube/projet-etude`, ce qui sert directement la recette `just kubeconfig`.

Le `devShell` (*nixos/flake.nix:72-93*) embarque l'intégralité de l'outillage opérationnel : `just`, `kubectl`, `helm`, `argocd`, `k9s`, `jq`, `yq-go`, `openssh`, `sops`, `age`, `ssh-to-age`, plus le package `projet-kubeconfig`. Aucun installateur n'est requis hors nix lui-même côté poste développeur : la commande `nix develop ./nixos` suffit à reproduire l'environnement d'opération.

4.2.4 Partitionnement via Disko

Disko (input flake) gère le partitionnement de manière déclarative. Le module *nixos/modules/disko-proxmox.nix* (58 lignes) déclare un layout GPT BIOS avec une partition `ext4` root unique, et active la console série pour Proxmox.

Le choix GPT BIOS plutôt qu'UEFI suit la convention du template Debian 12 `genericcloud` (mode legacy BIOS). Le filesystem `ext4` reste retenu pour sa stabilité éprouvée et son empreinte mémoire faible ; `btrfs` et `zfs` apportent le snapshot natif mais sont surdimensionnés pour un cluster K3s qui s'appuie sur un stockage `local-path` éphémère (cf. §4.4.7 et §2.6 pour la limite associée).

La console série activée par défaut sert deux usages : récupération en cas de réseau interrompu via la console Proxmox web ou `qm terminal`, et lecture des logs de boot pendant le debug initial du module *common.nix*.

Disko opère pendant le processus `nixos-anywhere` (cf. §4.2.5) : le disque cible est repartitionné selon la déclaration *disko-proxmox.nix*, puis NixOS est installé sur la partition fraîchement créée.

4.2.5 Provisionnement par nixos-anywhere

Cinq scripts orchestrent le cycle de vie côté hôte Proxmox, tous regroupés sous *proxmox/*.

proxmox/create-template.sh (38 lignes) télécharge l'image cloud Debian 12, crée le template VMID 9000, configure cloud-init et l'agent *qemu-guest-agent*. Une seule exécution par installation Proxmox, idempotent au second appel.

proxmox/create-vm.sh (45 lignes) clone le template vers les VMs 301, 302, 303. Spécifications identiques : 2 vCPU, 6 144 MiB RAM, 32 GB disk, IP statiques 192.168.1.61, 192.168.1.62, 192.168.1.63.

proxmox/bootstrap-host-keys.sh (92 lignes) génère localement les SSH host keys ed25519 des trois VMs, persistées dans *secrets/host-keys/<host>/*. Le script aligne ensuite *nixos/.sops.yaml* avec les fingerprints age dérivés via *ssh-to-age* (cf. §4.2.6 pour la trust chain). Idempotent : aucune action si les clés existent déjà.

proxmox/install-nixos.sh (115 lignes) déploie NixOS sur les trois VMs en parallèle via *nixos-anywhere* exécuté par *xargs -P 3*. Le mécanisme *nixos-anywhere* repose sur **kexec** : il démarre un kernel Nix de provisionnement à chaud sur la machine cible (Debian en cours d'exécution au moment du lancement), partitionne via *Disko*, installe NixOS, redémarre. Les SSH host keys persistantes sont placées en extra-files lors de cette étape, ce qui rend *secrets.yaml* décryptable dès le premier boot NixOS.

proxmox/switch-nixos.sh (48 lignes) applique *nixos-rebuild switch* séquentiellement sur les trois nœuds, après modification du flake. La séquence évite de redémarrer simultanément les trois nœuds et préserve la disponibilité du cluster pendant l'opération.

Une limite assumée : *nixos-anywhere* est consommé depuis la branche master GitHub (*proxmox/install-nixos.sh:85*), sans pinning par tag ni commit. Pour un environnement de production, un pinning explicite serait à introduire (cf. §2.6, point résiduel équivalent côté reproductibilité).

4.2.6 Gestion des secrets via sops-nix

sops-nix est l'input flake qui intègre *sops* (Mozilla Secret OPerationS) au boot NixOS. Le mécanisme repose sur une trust chain à quatre recipients age, déclarés dans *nixos/.sops.yaml* :

- une clé age dev locale (poste développeur, *~/ .config/sops/age/keys.txt*) ;
- trois clés age dérivées des SSH host keys ed25519 des trois VMs, via *ssh-to-age*.

Le fichier *nixos/secrets/secrets.yaml*, chiffré avec ces quatre recipients, contient trois secrets : *k3s_token* (jeton de jonction du cluster K3s), *discord_webhook_url* (webhook Alertmanager, cf. §4.8.4), *argocd_image_updater_ssh_key_b64* (clé SSH read-write utilisée par *argocd-image-updater* pour le write-back Git, cf. §4.4.6).

Le déchiffrement au boot fonctionne ainsi : *nixos/modules/secrets.nix* (106 lignes) déclare via *sops-nix* les chemins runtime des secrets ; chaque nœud utilise sa SSH host key ed25519 locale, dérivée en clé age, pour déchiffrer *secrets.yaml*. Aucun mot de passe humain n'est saisi pendant l'installation ou les rebuilds. Le service *k3s-server* lit `tokenFile = config.sops.secrets.k3s_token.path` (*nixos/modules/k3s-server.nix:9-10*), Alertmanager monte le secret discord, Image Updater monte sa clé SSH.

La trust chain est rendue stable par l'étape *bootstrap-host-keys.sh* (cf. §4.2.5) : les SSH host keys sont générées une seule fois côté développeur, persistées dans le dépôt (chiffré), puis placées sur disque lors du *nixos-anywhere*. Elles survivent à tout redéploiement ultérieur. Ce dispositif élimine le problème classique de bootstrap des secrets : pas de chicken-and-egg, aucune action manuelle requise sur les nœuds après leur création.

La rotation des secrets passe par `just sops-rotate`, qui appelle `sops updatekeys` pour re-chiffrer *secrets.yaml* avec les recipients courants de *.sops.yaml*. La cible interne est trimestrielle (cf. §3.1.3 et §3.5).

4.2.7 Exposition par Tailscale serve

L'exposition utilisateur ne passe pas par un Ingress public ou par cert-manager classique : **Tailscale serve** termine TLS directement sur le tailnet privé. Deux modules NixOS portent l'intégration.

nixos/modules/tailscale.nix (24 lignes) active `services.tailscale`, ajoute l'interface tailscale au firewall trusté, et déclare la pre-auth-key consommée par `tailscaled`. La clé est lue depuis *secrets/tailscale-authkey* côté poste développeur lors du *nixos-anywhere*.

nixos/modules/tailscale-serve.nix (62 lignes) déclare une option NixOS `projet.tailscale.serve.routes` qui mappe des ports HTTPS tailnet vers des backends NodePort locaux. Le module génère un service systemd qui appelle `tailscale serve` au boot avec la configuration consolidée.

Sur le control-plane (*nixos/hosts/k3s-cp-1.nix:13-28*), trois routes sont actives :

- 443 HTTPS tailnet vers NodePort 30443 (ArgoCD UI) ;
- 8443 HTTPS tailnet vers NodePort 30030 (Grafana) ;
- 9443 HTTPS tailnet vers NodePort 30880 (Traefik web, qui sert l'application via Ingress /app-demo).

Le mécanisme apporte trois avantages au regard du cahier (*docs/cahier-des-charges.pdf* §IV §4 sécurité) :

- TLS Let's Encrypt automatique géré par Tailscale, sans cert-manager ni renouvellement à orchestrer ;
- aucun port n'est exposé publiquement, l'accès reste conditionné à l'appartenance au tailnet ;
- la déclaration est centralisée dans le flake : ajouter ou retirer une route revient à éditer le fichier hôte et relancer `just switch`.

Limite assumée : l'URL effective dépend du nom DNS choisi par Tailscale (`<hostname>.<tailnet>.ts.net`), anonymisé dans le présent document (cf. §2.6). L'utilisateur final accède au service via cette URL, sans indirection supplémentaire.

4.3 Conteneurisation

4.3.1 Choix de l'image runtime distroless

Le runtime des trois conteneurs applicatifs (api, worker, audit-purge) repose sur `gcr.io/distroless/static-debian12:nonroot` (cf. *projet-etude-app-demo/Dockerfile:11,18,25*). Quatre alternatives ont été instruites avant ce choix.

- `alpine:3` (≈ 7 MB) : empreinte mémoire faible, mais musl libc et environnement shell présent par défaut, qui élargissent inutilement la surface d'attaque. Convient mal aux binaires Go avec CGO désactivé (musl ajoute du travail au lieu d'en retirer).
- `debian:12-slim` (≈ 80 MB) : compatibilité maximale, mais shell, apt et binaires GNU embarqués augmentent inutilement la surface d'attaque pour un binaire Go statique.
- `scratch` (0 MB) : aucune dépendance, mais aucun certificat racine, aucun `/etc/passwd` pour un user nommé, aucun `/etc/nsswitch.conf`. Fonctionnel pour les binaires qui n'ont pas besoin de TLS sortant, ce qui n'est pas le cas ici (l'API appelle Dex en OIDC, l'OTel exporter parle gRPC TLS vers Tempo).
- `gcr.io/distroless/static-debian12:nonroot` (≈ 2 MB) : aucun shell, aucun package manager, aucune libc dynamique, mais embarque les certificats racine, `/etc/passwd` avec un user `nonroot` UID 65532, et `/etc/nsswitch.conf` correct.

La variante `nonroot` est retenue plutôt que la variante par défaut. Elle force l'exécution sous UID 65532 sans configuration additionnelle côté Pod, et aligne par défaut sur le profil Pod Security Standards Restricted de Kubernetes.

Une limite assumée : l'absence de shell empêche `kubectl exec` interactif pour le debug. Le compromis est cohérent avec le périmètre projet : l'investigation passe par les logs JSON structurés, les métriques et les traces Tempo (cf. §4.8). Le coût d'inspection est reporté sur l'observabilité, où il est mieux instrumenté.

4.3.2 Dockerfile multi-stage à trois cibles

Le *Dockerfile* (29 lignes) embarque un seul stage de build et trois stages de runtime nommés, ce qui factorise la compilation et produit trois images dédiées sans duplication de code.

```
FROM golang:1.25-alpine AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -trimpath -o
/out/api ./cmd/api && \
    CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -trimpath -o
/out/worker ./cmd/worker && \
    CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -trimpath -o
/out/audit-purge ./cmd/audit-purge

FROM gcr.io/distroless/static-debian12:nonroot AS api
COPY --from=build /out/api /app
USER nonroot
EXPOSE 8080
```

ENTRYPOINT ["/app"]

stages worker (:8081) et audit-purge (sans port) identiques

Trois décisions structurent ce fichier. Le stage de build sépare la couche dépendances (COPY go.mod go.sum puis RUN go mod download) de la couche source (COPY . .), ce qui invalide le cache uniquement quand go.mod ou go.sum change : une modification d'un fichier .go reste très rapide à rebuild. Les trois binaires sont compilés dans un seul RUN, ce qui mutualise le téléchargement et l'analyse des modules entre composants. Les trois stages de runtime portent chacun le binaire correspondant et son port, et sont sélectionnés par la matrice CI via target: \${{ matrix.component }} (cf. [.github/workflows/build.yml:84-93](#)).

4.3.3 Optimisations binaire et taille d'image

Quatre options de build s'enchaînent dans la commande go build :

- CGO_ENABLED=0 désactive l'interface C, supprime toute dépendance dynamique à la libc, et produit un binaire entièrement statique. Cette condition est nécessaire pour cibler distroless/static, qui ne fournit pas de libc dynamique.
- GOOS=linux fixe explicitement la cible, ce qui évite les surprises en cas de cross-compilation depuis un poste macOS ou Windows.
- -trimpath remplace les chemins absolus du système de build par les paths des modules Go, ce qui rend le binaire reproductible bit-à-bit pour une même source, et supprime toute fuite de path développeur.
- -ldflags="-s -w" retire la table des symboles (-s) et les informations de debug DWARF (-w), pour une réduction de 20 à 30 % de la taille binaire sans perte fonctionnelle. La perte concerne le debug delve, qui reste possible en build local sans ces flags.

L'image finale par composant pèse 25 à 35 MB, dont environ 2 MB pour la couche distroless et 20 à 32 MB pour le binaire Go statique (runtime, std lib, dépendances directes). À titre de comparaison, un build équivalent sur alpine:3 avec libc-compat dépasse 80 MB ; sur debian:12-slim, 120 MB.

4.3.4 SecurityContext et durcissement runtime

Le durcissement runtime empile cinq mécanismes complémentaires, déclarés à la fois côté image et côté Pod.

Côté image, l'instruction USER nonroot (cf. *Dockerfile:13,20,27*) impose l'UID 65532 par défaut.

Côté Pod, le securityContext du Deployment (cf. *kubernetes/apps/projet-etude-app-demo/deployment-api.yaml:43-44, 46-47, 118-120*) déclare :

- runAsUser: 65532 et runAsGroup: 65532 : alignement explicite avec l'UID de l'image, qui empêche un montage de volume avec mauvais owner ;
- readOnlyRootFilesystem: true : le filesystem racine du conteneur est en lecture seule. Toute écriture passe par un emptyDir explicite, ce qui réduit le rayon d'action d'une compromission ;
- allowPrivilegeEscalation: false : un setuid ou un changement de capabilities en cours d'exécution est rejeté par le kernel ;

- `capabilities.drop: ["ALL"]` : l'application n'a besoin d'aucune capability Linux ; toutes sont retirées ;
- `seccompProfile.type: RuntimeDefault` : le profil seccomp du runtime container est appliqué, ce qui filtre les syscalls dangereux selon la politique containerd par défaut.

L'effet est mesurable : le job kube-score côté CI (cf. [.github/workflows/validate.yml](#)) ne signale aucun problème de securityContext sur les manifests applicatifs, à l'exception des quatre tests ignorés volontairement et justifiés en commentaire du workflow.

4.3.5 Stratégie de tag et de cache GHA

La stratégie de publication d'image privilégie l'immutabilité d'identification et la rapidité de rebuild.

Deux tags sont produits par `docker/metadata-action@v5` pour chaque image (cf. [.github/workflows/build.yml:75-83](#)) :

- `type=ref,event=branch` produit le tag `main` pour la branche principale. Ce tag reste mutable et utile pendant la mise au point, mais n'est jamais référencé dans les manifests Kubernetes en production.
- `type=sha,format=long` produit un tag `sha-<commit-sha-long>` qui identifie immuablement une révision exacte. C'est ce tag, ou plus précisément le digest associé, que l'argocd-image-updater détecte et écrit dans `kustomization.yaml` (cf. §4.4.6).

Le rollback applicatif consiste à reverter le commit Git qui modifie le digest dans la kustomization, ce qui force ArgoCD à redéployer le digest précédent. Aucune commande `kubectl rollout undo` n'est nécessaire.

Le cache buildx repose sur GitHub Actions Cache, scope par composant (cf. [.github/workflows/build.yml:92-93](#)) :

```
cache-from: type=gha,scope=${{ matrix.component }}
cache-to:   type=gha,mode=max,scope=${{ matrix.component }}
```

Trois caches indépendants sont maintenus (`api`, `worker`, `audit-purge`). L'option `mode=max` exporte toutes les couches intermédiaires, ce qui maximise les hits sur builds successifs ; le scope par composant évite qu'un changement de code touchant uniquement l'api n'invalide le cache du worker ou de l'audit-purge.

L'effet observé : sur un rebuild incrémental (changement d'un fichier `.go` côté api seul), le job `build` du composant `api` retombe autour de 1 minute (vs 4 à 5 minutes en cold cache), et les jobs `worker` et `audit-purge` restent intacts en cache.

4.4 Orchestration Kubernetes via K3s

4.4.1 Choix de K3s

K3s est une distribution Kubernetes packagée par Rancher (désormais SUSE), conçue pour les environnements à empreinte contrainte (edge, IoT, bare-metal restreint). L'API reste 100 % compatible Kubernetes : CRDs, kubectl, RBAC, NetworkPolicies, ServiceAccounts fonctionnent à l'identique. La différence se situe dans l'emballage et le dimensionnement.

Trois caractéristiques motivent le choix.

Empreinte mémoire faible. Le control-plane K3s consomme environ 150 Mo de RAM en idle, contre 400 Mo pour un Kubernetes vanilla (kubelet + kube-apiserver + controller-manager + scheduler + kube-proxy en processus séparés). Sur une VM de 6 GiB, le différentiel libère plus de 4 % de mémoire utile, qui revient à l'application.

Binaire unique. K3s s'installe via un seul exécutable ($\approx$ 50 MB), qui contient tous les composants serveur ou agent selon le rôle. Le mode déclaratif NixOS active simplement `services.k3s` avec `role = "server"` ou `role = "agent"` (cf. *nixos/modules/k3s-server.nix:7-10*, *nixos/modules/k3s-agent.nix*).

Composants embarqués par défaut. K3s livre Traefik (ingress), Flannel (CNI), CoreDNS, ServiceLB et local-path-provisioner, ce qui supprime l'installation manuelle de ces briques. Chaque composant peut être désactivé par un flag, ce qui évite l'empilement non désiré (cf. §4.4.2).

Alternatives écartées plus en détail qu'en §2.3. Kubernetes vanilla via kubeadm reste possible mais ajoute environ 250 Mo de RAM en idle et augmente la complexité opérationnelle. k0s offre une empreinte comparable à K3s mais bénéficie d'une adoption plus restreinte et d'une compatibilité moins éprouvée avec ArgoCD et la stack Prometheus. kind et minikube sont conçus pour le poste développeur (cluster en conteneur ou en VM single-node), pas pour un cluster multi-nœuds persistant.

4.4.2 Topologie et flags k3s

Le cluster réunit un control-plane (*k3s-cp-1*, 192.168.1.61) et deux workers (*k3s-worker-1* sur 192.168.1.62, *k3s-worker-2* sur 192.168.1.63). Le dimensionnement à 1+2 correspond au seuil minimal qui tolère la perte d'un worker tout en laissant deux nœuds capables de planifier des Pods. Aucune haute disponibilité du control-plane n'est mise en place : le projet d'études n'en a pas le besoin, et la limite est documentée (cf. §2.6).

Cinq flags sont passés au serveur K3s (cf. *nixos/modules/k3s-server.nix:11-23*) :

- `--node-ip=${config.projet.k3s.nodeIp}` et `--advertise-address=${config.projet.k3s.nodeIp}` : fixent explicitement l'IP du nœud sur le LAN privé, ce qui évite l'auto-détection d'une IP tailnet ;
- `--write-kubeconfig-mode=0644` : rend le kubeconfig lisible par l'utilisateur ops, condition nécessaire à la récupération via SSH par `just kubeconfig` ;
- `--disable=serviceLB` : ServiceLB allouerait des IPs externes aux Services de type LoadBalancer. Sans cloud provider et avec Traefik intégré comme ingress, ce composant n'a aucun rôle ; le désactiver libère $\sim$ 30 Mo de RAM ;

- `--disable-cloud-controller` et `--kubelet-arg=cloud-provider=` : sans la désactivation explicite du cloud-controller-manager, K3s tente de l'instancier en boucle (fenêtre de démarrage trop étroite face au bootstrap RBAC sur K3s 1.34, ce qui le fait crasher). Sans le flag kubelet associé, le taint `node.cloudprovider.kubernetes.io/uninitialized` resterait sur les nœuds et empêcherait toute planification. La justification est inscrite en commentaire dans *nixos/modules/k3s-server.nix:14-22*.

Les agents joignent le control-plane via le token K3s lu depuis sops (`tokenFile = config.sops.secrets.k3s_token.path`, cf. *nixos/modules/k3s-agent.nix*).

Côté réseau, **Flannel** sert de CNI avec un overlay VXLAN par défaut. Suffisant pour un cluster bare-metal à trois nœuds, sans la complexité d'un Cilium ou d'un Calico qui n'apporteraient pas de fonction nouvelle dans ce périmètre (eBPF observabilité couvert par Kepler, NetworkPolicies natives suffisantes).

Côté stockage, le provisioner local-path écrit les PVCs dans `/var/lib/rancher/k3s/storage/` sur le nœud où le Pod est planifié. Aucune réplication entre nœuds. La limite est documentée et constitue un point résiduel (cf. §2.6).

4.4.3 Cartographie des namespaces

Huit namespaces plateforme et application sont créés par le pipeline GitOps, un par concern. La séparation rend lisibles à la fois la RBAC (un rôle par namespace), les NetworkPolicies (isolation par défaut), et les quotas éventuels.

Namespace	Composants	Rôle
argocd	ArgoCD, Image Updater	Contrôleur GitOps + écriture digest
monitoring	Prometheus, Loki, Tempo, Alloy, Grafana, Alertmanager	Métriques, logs, traces, dashboards, alerting
security	Vault DEV, Dex OIDC	Coffre-fort runtime + fournisseur d'identité
external-secrets	ESO operator + webhook	Synchronisation Vault vers Secrets K8s
trivy-system	trivy-operator	Scan continu CVE + ConfigAudit + RbacAssessment
kepler	Kepler DaemonSet	Mesure énergétique par pod
cnpg-system	CloudNative-PG operator	Gestion des clusters Postgres
projet-etude-app-demo	api, worker, audit-purge, ticketdb, NetworkPolicies, HPA, PDB, dashboards	Application métier

Aux huit s'ajoutent les namespaces système (`kube-system` pour Traefik, CoreDNS, local-path-provisioner, metrics-server ; `kube-public`, `kube-node-lease`, `default`). Les manifests applicatifs n'écrivent jamais dans les namespaces système.

4.4.4 Bootstrap déclaratif via k8s-bootstrap.nix

Le bootstrap d'ArgoCD est entièrement déclaratif et précède toute action manuelle. Le module *nixos/modules/k8s-bootstrap.nix* (133 lignes) écrit, pendant le *nixos-rebuild*, les manifests YAML nécessaires dans */var/lib/rancher/k3s/server/manifests/*. K3s applique automatiquement ces manifests au démarrage du control-plane.

Quatre éléments sont posés à cette étape :

- la création du namespace *argocd* ;
- l'installation d'ArgoCD via les manifests upstream consommés depuis l'input flake *argo-cd* épinglé à *v2.13.0* (cf. *nixos/flake.nix:10-13*), ce qui supprime toute dépendance à une release Helm ou à un installeur externe ;
- l'exposition du service *argocd-server* en NodePort 30443, pour permettre à Tailscale serve d'y router le port 443 tailnet (cf. §4.2.7) ;
- la **root application** ArgoCD, qui pointe vers le dossier *kubernetes/applications/* du dépôt infra. Cette root app applique le pattern *app-of-apps* : elle déploie d'autres Application ArgoCD, qui à leur tour déploient les composants plateforme.

À partir du premier boot du control-plane, ArgoCD est posé et la root app commence à synchroniser le contenu Git, sans aucune intervention humaine. Le mot de passe initial admin reste extractible par `just argocd-password`.

4.4.5 ApplicationSet GitOps multi-catégorie

L'**ApplicationSet** (*kubernetes/applications/platform/apps-applicationset.yaml*) est une ressource ArgoCD qui génère dynamiquement des Application à partir d'un générateur. Le générateur Git scanne *kubernetes/apps/* et crée une Application par dossier (avec exclusion des dossiers commençant par `_`).

Le template applique pour chaque dossier :

- `name = {{.path.basename}}` : nom de l'app = nom du dossier (par exemple *projet-etude-app-demo*) ;
- `source.path = {{.path.path}}` : chemin du dossier Kustomize ;
- `destination.namespace = {{.path.basename}}` : namespace de destination identique au nom du dossier (créé via *CreateNamespace=true*) ;
- `syncPolicy.automated: { prune: true, selfHeal: true }` : suppression auto des ressources orphelines, restauration auto des ressources modifiées hors GitOps ;
- annotations *argocd-image-updater* pour le digest pinning automatisé (cf. §4.4.6).

L'effet pratique : ajouter une nouvelle application métier au cluster revient à créer un dossier *kubernetes/apps/<nom>/* avec un *kustomization.yaml*. L'ApplicationSet la détecte, crée l'Application correspondante, et lance la synchronisation. Aucune modification de la configuration ArgoCD ni du module Nix n'est nécessaire.

Les Applications plateforme (monitoring, security, observability, data, platform) suivent un schéma plus simple : un fichier par composant dans *kubernetes/applications//*, qui définit directement la ressource Application pointant vers un chart Helm ou un dossier Kustomize. La root app du module *k8s-bootstrap.nix* couvre l'ensemble.

4.4.6 Image Updater write-back Git et digest pinning

L'image-updater (chart `argocd-image-updater` 0.12.0, cf. `kubernetes/applications/platform/argocd-image-updater.yaml:13`) automatise la propagation des nouveaux builds vers le cluster, sans bypass du flux GitOps. Le mécanisme tient en quatre étapes.

L'image-updater surveille périodiquement le registry GHCR pour les images déclarées dans l'annotation `argocd-image-updater.argoproj.io/image-list` des Applications. Les images suivies sont, par convention de l'ApplicationSet (`kubernetes/applications/platform/apps-applicationset.yaml:27-32`) :

```
argocd-image-updater.argoproj.io/image-list:  
api=ghcr.io/<org>/{{.path.basename}}-  
api:main,worker=ghcr.io/<org>/{{.path.basename}}-worker:main  
argocd-image-updater.argoproj.io/api.update-strategy: digest  
argocd-image-updater.argoproj.io/worker.update-strategy: digest  
argocd-image-updater.argoproj.io/write-back-method: git  
argocd-image-updater.argoproj.io/git-branch: main  
argocd-image-updater.argoproj.io/write-back-target: kustomization
```

La stratégie `digest` impose le suivi du digest SHA exact derrière le tag `main`, et non du tag mutable. Lorsqu'un nouveau build pousse une image taguée `main` vers GHCR (cf. §4.5.2), l'image-updater résout le digest associé.

La méthode de `write-back git` indique que la mise à jour passe par un commit Git, et non par une modification directe de la ressource Application ArgoCD. La cible `kustomization` désigne le fichier `kustomization.yaml` du dossier de l'application. L'image-updater édite la section `images:` de ce fichier pour y inscrire le nouveau digest.

ArgoCD, qui scrute le dépôt Git par défaut toutes les trois minutes, détecte le commit, déclenche une nouvelle sync, et applique le manifest mis à jour au cluster. Le rolling update du Deployment associé démarre, sous contrainte du PDB (cf. §4.4.8).

La clé SSH `read-write` utilisée par l'image-updater pour pousser ce commit est lue depuis `sops` (`argocd_image_updater_ssh_key_b64`, cf. §4.2.6) et montée en `Secret` dans le namespace `argocd`. Aucune clé Git n'est saisie manuellement.

Trois propriétés découlent de ce dispositif. La reproductibilité est totale : chaque déploiement applicatif est identifié par un digest immuable, inscrit dans Git, qui ne dépend plus de la mutabilité du tag `main`. Le rollback applicatif consiste à reverter le commit qui a modifié le digest ; ArgoCD redéploie alors le digest précédent. La traçabilité est complète : l'historique Git du dépôt infra reflète chaque mise à jour d'image, avec auteur (image-updater), date et message normalisé.

4.4.7 Ressources de l'application

Le dossier `kubernetes/apps/projet-etude-app-demo/` regroupe 20 manifests, 1 433 lignes au total, qui décrivent l'ensemble des ressources Kubernetes de l'application. Ils sont organisés ici par concern fonctionnel ; les détails relevant de la sécurité, de l'observabilité, de la résilience et de l'application figurent dans les sections dédiées.

Déploiement et services. `deployment-api.yaml` (Deployment api, spread anti-affinity, securityContext durci, probes, env vars), `deployment-worker.yaml` (idem pour le worker, probes sur le port metrics

8081), *service.yaml* (ClusterIP api sur port 80 vers 8080), *service-worker.yaml* (Service Headless pour le scrape par-pod du worker).

Scaling et résilience. *hpa.yaml* (HPA api 2-8 sur CPU 60 %), *hpa-worker.yaml* (HPA worker 1-8 sur CPU 70 %, métrique `jobs_queue_depth` laissée en commentaire pour une évolution future), *pdb.yaml* (PodDisruptionBudget api `minAvailable: 1`, worker `maxUnavailable: 1`). Détail en §4.9.

Données. *database.yaml* (CNPG Cluster `ticketdb`, Postgres 16.4, instance unique, PVC 1 Gi local-path). Limite à instance unique documentée en §2.6.

Réseau et exposition. *ingress.yaml* (Ingress Traefik /app-demo plus Middleware StripPrefix, qui retire le préfixe avant routage vers le Service api), *networkpolicy.yaml* (quatre NetworkPolicies dédiées api, worker, audit-purge et chaos-killer). Détail réseau et sécurité en §4.7.

Sécurité et secrets. *externalsecrets.yaml* (trois ExternalSecrets `oidc-client`, `app-session`, `app-webhook`, qui synchronisent depuis Vault), *chaos.yaml* (ServiceAccount, Role et RoleBinding scoped pour le chaos-killer, plus le CronJob suspendu par défaut). Détail en §4.7.

RGPD et maintenance. *cronjob-audit-purge.yaml* (CronJob hebdomadaire `15 3 * * 0` qui exécute `cmd/audit-purge` avec une rétention de 90 jours par défaut). Détail en §4.6.5.

Observabilité. *servicemonitor.yaml* et *servicemonitor-worker.yaml* (scrape Prometheus toutes les 15 secondes), *prometheusrule.yaml* (quatre recording rules plus cinq alerting rules dont SLO multi-burn-rate), *dashboard.yaml*, *dashboard-slo.yaml*, *dashboard-worker.yaml* (trois dashboards Grafana injectés en ConfigMaps avec le label `grafana_dashboard: "1"`, importés automatiquement par le sidecar Grafana). Détail en §4.8.

Le fichier d'entrée *kustomize kustomization.yaml* liste l'ensemble des manifests, et déclare trois images (api et worker en stratégie digest, audit-purge en stratégie tag puisque le CronJob hebdomadaire tolère le retard d'une mise à jour cosmétique).

4.4.8 Probes et stratégie de déploiement

Les probes sont calibrées par composant.

Le Pod api expose trois probes sur le port 8080 : `startupProbe` sur `/readyz` avec un budget large (jusqu'à 30 secondes pour atteindre Ready après démarrage), `readinessProbe` sur `/readyz` toutes les 5 secondes pour gérer les pics de charge et les déconnexions Postgres, `livenessProbe` sur `/healthz` toutes les 10 secondes pour détecter un blocage du goroutine principal.

Le Pod worker expose deux probes sur le port 8081 (port metrics) : `readinessProbe` sur `/readyz` toutes les 5 secondes et `livenessProbe` sur `/healthz` toutes les 10 secondes. Le worker n'a pas de phase de démarrage longue, le `startupProbe` est inutile.

Grafana (chart `kube-prometheus-stack`) utilise les défauts du chart : `startupProbe` jusqu'à 5 minutes pour tolérer le chargement initial des dashboards, `livenessProbe` toutes les 30 secondes, `readinessProbe` toutes les 10 secondes.

Côté stratégie de déploiement, quatre mécanismes empilés assurent l'absence d'interruption.

La stratégie par défaut `RollingUpdate` (aucune surcharge dans les Deployments) remplace progressivement les Pods, en respectant `maxUnavailable: 25 %` et `maxSurge: 25 %`. Sur deux répliques api, un seul Pod est indisponible à la fois.

Le `PodDisruptionBudget` (cf. *kubernetes/apps/projet-etude-app-demo/pdb.yaml*) garantit qu'un drain volontaire (`kubectl drain`) ne descend jamais sous le seuil minimal de Pods sains. Pour l'api, `minAvailable: 1` ; pour le worker, `maxUnavailable: 1`.

L'HPA ajuste le nombre de répliques à la charge observée. Les politiques de scale-up et scale-down sont configurées pour absorber les pics : scale up de +2 répliques toutes les 15 secondes, scale down de -1 réplique toutes les 60 secondes, ce qui rend l'application réactive aux montées en charge sans osciller (cf. §4.9.1).

ArgoCD avec `prune: true` et `selfHeal: true` rétablit toute dérive du manifest sous une minute. Une suppression manuelle d'un `ConfigMap`, d'un `Service` ou d'un `PDB` est rétablie sans intervention humaine, ce qui élimine la possibilité d'un état désynchronisé entre le dépôt Git et le cluster.

4.5 CI/CD

4.5.1 Choix de GitHub Actions et de GHCR

La chaîne CI/CD repose sur GitHub Actions pour l'exécution des pipelines et sur GHCR (GitHub Container Registry) pour la livraison des images. Le comparatif global figure en §2.3 ; cette section détaille les raisons techniques qui ont conduit à écarter les alternatives.

GitHub Actions cumule cinq propriétés cohérentes avec le périmètre :

- intégration native avec le dépôt Git, ce qui supprime la configuration d'un déclencheur webhook entre un SaaS CI tiers et GitHub ;
- runners hébergés gratuits avec quotas généreux pour les dépôts publics ;
- configuration en YAML déclaratif, ce qui s'aligne avec la philosophie globale du projet ;
- catalogue d'actions tiers stable, qui couvre les besoins ici utilisés (`actions/checkout`, `actions/setup-go`, `docker/setup-buildx-action`, `docker/build-push-action`, `aquasecurity/trivy-action`, `golangci/golangci-lint-action`, `actions/upload-artifact`) ;
- aucune infrastructure de runner à maintenir, ce qui économise le temps d'administration que demanderait Jenkins self-hosted ou GitLab Runner.

GitLab CI a été écarté pour deux raisons : il imposerait soit une instance GitLab self-hosted à maintenir (overhead d'administration), soit le passage sur SaaS `gitlab.com` avec un coût récurrent sur dépôts privés. Jenkins a été écarté pour sa configuration impérative (Groovy DSL) et l'overhead de maintenance d'un controller plus des agents.

GHCR (`ghcr.io`) sert de registry d'images. Quatre propriétés motivent ce choix face à un MinIO ou à un registry on-prem :

- authentification native via `GITHUB_TOKEN`, sans clé long-lived à provisionner ;
- aucun service à maintenir (pas de stockage, pas de TLS à orchestrer, pas de PVC à dimensionner) ;
- coût nul pour les dépôts publics et quotas généreux côté privé ;
- intégration directe avec `docker/build-push-action` et `argocd-image-updater`, qui consomment GHCR sans configuration spécifique.

Limites assumées : un certain *vendor lock-in* GitHub. La portabilité reste préservée puisque les manifests Kubernetes, le Dockerfile et le code Go ne dépendent en rien de GitHub Actions ; un basculement vers un autre CI consisterait à réécrire deux fichiers YAML.

4.5.2 Pipeline application

Le pipeline application (`.github/workflows/build.yml`, 113 lignes) enchaîne trois jobs sur push main et sur pull request.

Le job `lint` (cf. `.github/workflows/build.yml:17-28`) installe Go 1.25 avec cache, puis lance `golangci-lint-action@v7` en version `v2.6.0` avec un timeout de 5 minutes. Les linters activés couvrent les classiques (`errcheck`, `govet`, `staticcheck`, `gosimple`, `ineffassign`, `unused`), avec configuration projet dans `projet-etude-app-demo/.golangci.yml`.

Le job `test` (cf. `.github/workflows/build.yml:30-57`) exécute `go vet` puis la commande de test :

```
go test -race -count=1 -timeout=300s -coverprofile=coverage.out -
covermode=atomic ./...
```

L'option `-race` active le détecteur de courses Go, indispensable pour un binaire qui traite des requêtes concurrentes et qui consomme des notifications LISTEN/NOTIFY Postgres. `-count=1` empêche le cache Go de tests de masquer un test flaky. `-covermode=atomic` est requis pour combiner couverture et `-race`. Les tests d'intégration utilisent `testcontainers-go` (cf. §4.6.8) et démarrent un Postgres 16-alpine éphémère par cas de test, ce qui requiert un démon Docker disponible sur le runner GitHub-hosted (présent par défaut sur les images `ubuntu-latest`).

Après la commande de test, le job génère un résumé GitHub via le `Step Summary` (couverture totale en pourcentage, plus détail par paquet), produit un rapport HTML `coverage.html`, et publie un artifact `coverage-report` qui contient `coverage.out` et `coverage.html`.

Le job `build` (cf. `.github/workflows/build.yml:59-112`) dépend des deux précédents (`needs: [lint, test]`) et ne s'exécute que sur push de main (`if: github.event_name == 'push' && github.ref == 'refs/heads/main'`). Il s'appuie sur une matrice à trois composants :

```
strategy:
  matrix:
    component: [api, worker, audit-purge]
```

Trois exécutions parallèles couvrent les trois cibles du Dockerfile. Chaque exécution enchaîne :

1. checkout et configuration de buildx ;
2. login GHCR via `GITHUB_TOKEN` (permissions `packages: write` déclarées au niveau workflow) ;
3. génération des tags par `docker/metadata-action@v5` (cf. §4.3.5) ;
4. build *load locally* (`load: true`) avec `target: ${{ matrix.component }}`, sans push, qui rend l'image disponible au démon Docker du runner pour le scan ;
5. scan Trivy bloquant : `aquasecurity/trivy-action@v0.36.0`, `exit-code: "1"`, `ignore-unfixed: true`, `severity: CRITICAL,HIGH`, qui fait échouer le job si une CVE non corrigée bloquante est détectée ;
6. push de l'image vers GHCR uniquement si le scan passe.

Le découplage entre build local et push permet de scanner avant publication, ce qui évite d'introduire dans GHCR une image vulnérable même temporairement. Le cache GHA est scope par composant (cf. §4.3.5).

4.5.3 Pipeline infrastructure

Le pipeline infrastructure (*.github/workflows/validate.yml*, 87 lignes) enchaîne six jobs parallèles. Tous restent purement déclaratifs : aucun cluster n'est instancié, aucun manifest n'est appliqué. La CI valide la syntaxe, la conformité aux schémas et le respect des conventions.

yamllint (ibiqlik/action-yamllint@v3) valide la syntaxe et la lisibilité des manifests YAML du dossier *kubernetes/*, selon les règles déclarées dans *.yamllint*.

actionlint (raven-actions/actionlint@v2) inspecte les workflows GitHub Actions eux-mêmes, ce qui détecte les erreurs de syntaxe, les expressions invalides et les usages dépréciés des actions.

shellcheck (ludeeus/action-shellcheck@2.0.0) analyse les cinq scripts shell du dossier *proxmox/* (*create-template.sh*, *create-vms.sh*, *bootstrap-host-keys.sh*, *install-nixos.sh*, *switch-nixos.sh*). Il signale les patterns d'erreur classiques (variables non quotées, redirections fragiles, exit codes non vérifiés).

kubeconform (version 0.6.7) valide les manifests Kubernetes contre les schémas officiels et contre les CRDs externes (récupérées via le catalogue Datree CRDs, cf. *.github/workflows/validate.yml:48-50*). Le mode *-strict* rejette les champs inconnus, ce qui force la déclaration explicite des CRDs (PrometheusRule, Application ArgoCD, Cluster CNPG, ExternalSecret, etc.).

nix-check exécute `nix flake check --no-build --show-trace` à la racine de *nixos/*. La commande évalue les trois configurations NixOS (*k3s-cp-1*, *k3s-worker-1*, *k3s-worker-2*) sans construire les dérivations, ce qui valide la cohérence du flake et de ses modules.

kube-score (version 1.20.0) analyse les manifests applicatifs après expansion Kustomize. Quatre tests sont volontairement ignorés, chacun documenté en commentaire dans le workflow :

- *container-image-pull-policy* : le pin par digest rend *IfNotPresent* correct ;
- *container-security-context-user-group-id* : *bitnami/kubect1*, utilisé par le *CronJob audit-purge*, impose un user fixe ;
- *container-security-context-readonlyrootfilesystem* : *kubect1* écrit dans *~/.kube/cache* au runtime ;
- *pod-networkpolicy* : faux positifs sur les *CronJobs* déjà couverts par les *NetworkPolicies* dédiées.

L'ensemble des six jobs s'exécute en parallèle. Le temps total du pipeline *validate.yml* reste donc cadré par le job le plus lent (typiquement *nix-check* ou *kubeconform*).

4.5.4 Triggers, artefacts et secrets

Les deux workflows partagent les mêmes triggers :

- `push` sur la branche `main` ;
- `pull_request` sur toutes les branches.

Sur PR, les jobs s'exécutent mais le job `build` côté application est court-circuité par la condition `if` qui restreint le push aux commits sur `main`. La PR voit donc la sortie de `lint` et `test` (et des six jobs `infra`), ce qui suffit à décider du merge.

Côté artefacts :

- les images Docker `ghcr.io/<org>/projet-etude-app-demo-{api,worker,audit-purge}:main` et `:sha-<long>` (rétention GHCR sans limite tant qu'un manifest le référence) ;
- le rapport de couverture HTML et le fichier `coverage.out` (artifact GitHub Actions, rétention 90 jours par défaut) ;
- le résumé de couverture inscrit dans le `Step Summary` du run GitHub, accessible directement depuis l'interface web.

Côté secrets :

- `GITHUB_TOKEN` (token intégré), avec les permissions explicitement déclarées au niveau workflow (contents: `read`, packages: `write`). Cette portée minimale autorise le push GHCR et interdit toute écriture sur le contenu du dépôt.
- aucun secret tiers n'est requis en CI. Les secrets Discord (webhook Alertmanager), Tailscale (pre-auth-key) et SSH (clés de déploiement et clé Image Updater) restent côté machine de déploiement (sops-nix pour le cluster, `secrets/` gitignored pour le poste).

Les jobs `lint`, `test` et `build` côté application, et les six jobs côté infrastructure, sont déclarés comme *required status checks* dans la configuration GitHub. Un check rouge bloque le merge sans dérogation possible via l'interface web.

4.5.5 Métriques CI observées

Les valeurs ci-dessous proviennent du snapshot du 24 mai 2026. Elles sont reproductibles via les commandes `gh run list` fournies en fin de section.

Pipeline `validate.yml` : sur les 25 derniers runs analysés, 24 succès et 1 échec, soit un taux de succès d'environ 96 %. La durée typique s'établit entre 33 et 41 secondes (calculée comme `delta startedAt → updatedAt`). La performance reste stable, car les jobs déclaratifs ne dépendent ni de runtime applicatif ni de tests d'intégration.

Pipeline `build.yml` : sur les 15 derniers runs analysés, 8 succès et 7 échecs, soit un taux de succès d'environ 53 %. Cette valeur reste biaisée par les itérations de mise au point initiale de `golangci-lint v2` (`golangci/golangci-lint-action@v7`, version `v2.6.0`) et de l'action `Trivy` (`aquasecurity/trivy-action@v0.36.0`), qui ont chacune introduit une rupture de format de configuration au moment de leur adoption. Sur les quatre derniers runs avant le snapshot, le taux atteint 100 %. La durée typique varie de 1 à 5 minutes, dominée par le test `race` et les trois builds en matrice.

Deploy time bout en bout, depuis `git push` jusqu'à pod Ready :

- pipeline `build.yml` côté application : 1 à 5 minutes selon l'état du cache GHA ;
- détection du nouveau digest GHCR par `argocd-image-updater` : intervalle de polling par défaut de 2 minutes ;
- write-back Git et re-sync ArgoCD : moins d'une minute ;
- rolling update Deployment (2 répliques api, RollingUpdate par défaut, PDB respecté) : moins de 30 secondes.

Total observé : 3 à 5 minutes en moyenne. La cible §2.2.1 (sous 5 minutes) est atteinte sur le snapshot, sous condition d'un cache GHA chaud.

Commandes de récupération à jour, exécutables sur tout poste avec gh authentifié :

```
# Infrastructure (validate.yml)
gh run list -R <org>/projet-etude-M1 --workflow=validate.yml --limit 100 \
  --json conclusion,startedAt,updatedAt | jq
```

```
# Application (build.yml)
cd projet-etude-app-demo
gh run list --workflow=build.yml --limit 100 \
  --json conclusion,startedAt,updatedAt | jq
```

Le KPI couverture de tests Go n'est pas figé dans le présent DAT. Il reste extractible via `go test -cover ./...` au jour de soumission, comme indiqué en §4.6.8.

4.6 Application métier

4.6.1 Choix de Go 1.25 et de HTMX

L'application métier sert de support de démonstration à la plateforme : un portail de gestion de tickets, pensé pour illustrer toute la chaîne (rendu HTML, persistance Postgres, jobs asynchrones, audit log, observabilité). Le choix technologique tient en deux décisions complémentaires : **Go 1.25** côté serveur, **HTMX** côté navigateur.

Go 1.25.0 (cf. `projet-etude-app-demo/go.mod:3`) coche cinq propriétés essentielles pour ce périmètre.

Le binaire est statique et compatible distroless (cf. §4.3). La compilation avec `CGO_ENABLED=0` produit un exécutable sans dépendance à la libc, condition nécessaire pour `gcr.io/distroless/static-debian12:nonroot`.

La bibliothèque standard couvre directement le besoin : `net/http` pour le serveur HTTP, `html/template` pour le rendu, `database/sql` plus le driver `pgx` pour Postgres, `log/slog` pour le logging structuré. Aucun framework HTTP n'est requis, ce qui supprime une dépendance lourde au profit d'une dépendance contrôlée.

La concurrence native (goroutines, channels) s'aligne avec le besoin du worker LISTEN/NOTIFY (cf. §4.6.4). Le détecteur de courses `-race` (cf. §4.5.2) valide en CI l'absence de course mémoire, ce qui est rare dans les autres environnements.

L'outillage qualité est mature : `golangci-lint` agrège une vingtaine de linters, `go test` intègre nativement le coverage et le race detector, `testcontainers-go` (cf. §4.6.8) gère les tests d'intégration avec un Postgres réel.

Go est largement adopté pour les outils CNCF (Kubernetes, ArgoCD, Helm, Prometheus, Loki, Tempo, CNPG). L'équipe travaille donc dans le même langage que l'infrastructure qu'elle opère, ce qui mutualise la montée en compétence.

Alternatives écartées : Java/Spring Boot impose la JVM dans le conteneur, soit ~200 MB d'image minimale et un cold start lent (GraalVM existe mais complexifie le pipeline de build). Node.js/TypeScript embarque un runtime de plusieurs centaines de MB, et offre des garanties de typage moins strictes au compile-time. Python/FastAPI réclame également un runtime interpréteur dans l'image, et l'écart de performance reste mesurable sur les charges HTTP soutenues. Rust offre des garanties équivalentes à Go mais coûte plus cher en temps d'apprentissage, dans un projet à durée bornée.

HTMX est retenu côté navigateur pour éviter une SPA séparée. Le serveur Go rend du HTML via `html/template`, et HTMX déclare via des attributs `hx-*` les interactions asynchrones (typiquement `hx-post`, `hx-target`, `hx-swap`). Aucune chaîne de build front-end n'est nécessaire : pas de webpack, pas de Node dans la CI, pas de bundler à maintenir. Le templating sert directement les fragments HTML que HTMX vient remplacer dans le DOM. Le compromis fonctionne bien pour le scope démo (CRUD de tickets, mises à jour partielles, formulaires) et reste cohérent avec l'objectif de sobriété de la plateforme.

4.6.2 Architecture `cmd/internal` et trois binaires

Le code respecte le layout standard Go.

```
projet-etude-app-demo/
├── cmd/
│   ├── api/main.go           (API HTTP :8080)
│   ├── worker/main.go       (worker async :8081 metrics)
│   └── audit-purge/main.go   (binaire one-shot, lancé par CronJob hebdo)
├── internal/
│   ├── auth/                (OIDC, sessions HMAC, RBAC 3 rôles)
│   ├── dbutil/              (ouverture pgxpool plus ping)
│   ├── httpapi/             (handlers, templates HTMX, endpoints chaos)
│   └── jobworker/           (LISTEN/NOTIFY plus polling plus retry)
└── backoff)
    ├── metrics/             (collecteurs Prometheus)
    ├── model/               (Ticket, Job, AuditEntry)
    ├── otelinit/            (initialisation OTLP gRPC tracer provider)
    ├── store/               (pgx, migrations, audit, jobs, tickets)
    └── testutil/postgres.go  (helper testcontainers)
```

Trois binaires distincts sont produits depuis ce code, chacun avec son propre point d'entrée dans `cmd/`.

Le binaire `api` (cf. `projet-etude-app-demo/cmd/api/main.go`) sert l'API HTTP sur le port 8080. Il initialise le tracer OTel (cf. `internal/otelinit/otelinit.go`), ouvre le pool `pgx`, exécute les migrations idempotentes (cf. `internal/store/store.go`), configure l'authentification OIDC, et démarre

`http.Server` avec `ReadHeaderTimeout: 5 * time.Second` pour limiter l'exposition aux slowloris.

Le binaire `worker` (cf. `cmd/worker/main.go`) tourne en tâche de fond, consomme la table `jobs` via `LISTEN/NOTIFY` plus polling de secours, et expose ses métriques Prometheus sur le port 8081. Il partage la majeure partie du code interne avec l'api (`pool pgx`, `store`, `metrics`, `otelinit`), mais ne sert aucune requête HTTP entrante.

Le binaire `audit-purge` (cf. `cmd/audit-purge/main.go`) est un exécutable one-shot lancé par le CronJob hebdomadaire (cf. §4.6.5). Il ouvre une connexion Postgres, exécute la requête de purge, et exit.

La séparation en trois binaires plutôt qu'un binaire monolithique avec sous-commandes apporte trois bénéfices concrets. Le scaling devient indépendant (HPA api 2 à 8 sur CPU, HPA worker 1 à 8 sur CPU, CronJob purge non scalé). Le `SecurityContext` peut être différencié par cible (l'audit-purge n'a besoin que d'un accès Postgres sortant ; l'api écoute un port). La matrice CI build les trois cibles en parallèle avec un cache GHA scope par composant (cf. §4.3.5).

Les paquets `internal/` ne sont importables que depuis le module lui-même, ce qui empêche tout couplage externe involontaire. La séparation `model` (types métier), `store` (persistance) et `httpapi` (transport) suit le clivage classique du domain-driven design léger.

4.6.3 Stockage Postgres via CloudNative-PG

Le stockage relationnel repose sur Postgres 16.4 géré par l'opérateur **CloudNative-PG** (CNPG, chart 0.22.1, cf. `kubernetes/applications/data/cloudnative-pg.yaml:13`). La ressource `Cluster` (CRD `postgresql.cnpg.io/v1`) est définie dans `kubernetes/apps/projet-etude-app-demo/database.yaml`.

L'opérateur prend en charge plusieurs concerns que l'on devrait sinon écrire à la main avec un `StatefulSet` vanilla. La création et la rotation des Secrets sont automatisées : CNPG génère un Secret `ticketdb-app` qui contient `DATABASE_URL` au format `pgx`, et l'expose au Deployment via une référence `secretKeyRef`. Le failover en cas de basculement primaire est géré automatiquement dans une configuration multi-instances (non activée ici, cf. ci-dessous). Le monitoring Prometheus est intégré : un `PodMonitor` est créé automatiquement (cf. `cloudnative-pg.yaml:19-20`), et un dashboard Grafana CNPG est posé dans le namespace `cnpg-system`.

La configuration retenue reste volontairement minimale pour le périmètre projet d'études :

- `instances: 1` (instance unique) ;
- `storage: { size: 1Gi, storageClass: local-path }` (PVC local-path) ;
- `monitoring: { podMonitorEnabled: true }`.

Les limites associées sont documentées en §2.6 : pas de réplication multi-instance, pas de backup formalisé (CNPG offre `backup.barmanObjectStore` natif mais celui-ci n'est pas activé), stockage local-path non répliqué entre nœuds.

Côté driver, l'application utilise `pgx/v5` (v5.9.2, cf. `go.mod:8`). `pgx` est le driver Postgres natif Go ; il supporte directement `LISTEN/NOTIFY`, expose un pool de connexions performant (`pgxpool`), et s'intègre nativement avec OpenTelemetry via `otelpgx` (v0.10.0, cf. `go.mod:7`). Toutes les requêtes

émises sur le pool génèrent un span dans Tempo, ce qui rend le profil d'accès BDD lisible côté observabilité (cf. §4.8).

Les migrations sont idempotentes et exécutées au démarrage de l'api dans `store.Migrate(ctx, pool)` (cf. *internal/store/store.go*). Les tables principales sont `tickets`, `jobs` (queue persistante) et `audit_log`. Le schéma `audit_log` est détaillé en §4.6.5 pour le volet RGPD.

4.6.4 Worker LISTEN/NOTIFY et propagation de trace

Le worker consomme une file de jobs persistés en base, sans broker externe (Kafka, RabbitMQ, NATS). La file est matérialisée par la table `jobs` ; la notification d'arrivée est portée par **LISTEN/NOTIFY** Postgres.

Le mécanisme tient en quatre points :

- côté écriture, l'api insère un job dans la table `jobs` (par exemple lors de la création d'un ticket qui doit déclencher une notification webhook), puis exécute `NOTIFY jobs_channel` ;
- côté lecture, le worker maintient une connexion `LISTEN jobs_channel` (cf. *internal/jobworker/worker.go*) qui reçoit les notifications en push ;
- en cas de notification ratée (déconnexion, redémarrage), un polling périodique de la table `jobs` rattrape les jobs en attente, ce qui rend le système fiable sans dédoublonner ;
- chaque exécution de job réussie marque la ligne `done`, chaque échec déclenche un retry avec backoff exponentiel ; un job kind inconnu est marqué `error` pour ne pas boucler.

L'absence de broker externe simplifie l'opération : pas de cluster Kafka à dimensionner, pas de gestion de partitions, pas de coordination ZooKeeper ou Raft. Pour le volume cible (quelques dizaines de jobs par jour en démo, quelques centaines en production réaliste), Postgres suffit. La contrepartie est la limite de débit (quelques milliers de jobs par seconde sur une instance Postgres standard), qui resterait à dépasser avant de migrer vers un broker dédié.

La propagation de trace fonctionne *cross-process*, du producteur (api) au consommateur (worker). L'api crée un span actif lors de la création du job, sérialise le contexte W3C TraceContext via le `TextMapPropagator OTel`, et stocke la sérialisation dans la colonne `jobs.trace_context` typée `JSONB` (cf. *internal/store/store.go:54*). Le worker, en consommant le job, désérialise le contexte et démarre un span `CONSUMER` lié au span parent (cf. *internal/jobworker/worker.go:140-152*). Le résultat dans Tempo est une trace continue qui couvre la requête HTTP qui a créé le job, l'écriture en base, la notification Postgres, le traitement worker, et l'éventuel appel webhook sortant. Ce dispositif rend le tracing distribué utilisable sans broker dédié.

Les métriques exposées par le worker incluent :

- `jobs_enqueued_total` (counter par kind), incrémenté par l'api côté écriture ;
- `jobs_processed_total` (counter par kind et outcome = ok ou error), incrémenté par le worker côté traitement ;
- `job_duration_seconds` (histogram par kind), avec exemplars `trace_id` ;
- `jobs_queue_depth` (gauge), publiée par un refresh périodique du nombre de jobs pending.

Le webhook sortant éventuel (variable d'environnement `WEBHOOK_URL`) cible une URL Discord (cf. §2.6 pour la limite RGPD associée) ou tout endpoint compatible avec le format Slack. La requête

HTTP sortante est elle aussi instrumentée OTel, ce qui prolonge la trace jusqu'à la réponse du webhook.

4.6.5 CronJob audit-purge et conformité RGPD

L'audit log applicatif est une table Postgres dédiée, alimentée à chaque action sensible (création de ticket, changement de rôle, login OIDC). Le schéma est défini dans *internal/store/store.go:64-82* et expose les colonnes suivantes :

- `ts` (timestamp), date de l'événement ;
- `user_sub` (texte), identifiant OIDC stable du sujet (sub claim) ;
- `user_email` (texte), email du sujet ;
- `ip` (INET), adresse IP de l'appelant ;
- `user_agent` (texte) ;
- `trace_id` (texte), identifiant trace OTel, qui rend chaque ligne corrélable à une trace Tempo ;
- `result` (texte), succès ou code d'erreur.

La purge automatique est portée par le binaire `audit-purge` (cf. *cmd/audit-purge/main.go*). Le binaire ouvre une connexion Postgres, exécute `DELETE FROM audit_log WHERE ts < now()` - `retention`, journalise le nombre de lignes supprimées, et exit. La rétention est configurable via la variable d'environnement `AUDIT_RETENTION_DAYS` (défaut 90 jours).

Le CronJob Kubernetes (cf. *kubernetes/apps/projet-etude-app-demo/cronjob-audit-purge.yaml:31-35*) déclenche ce binaire chaque dimanche à 03h15 UTC (`schedule: "15 3 * * 0"`), avec `restartPolicy: OnFailure` et un `backoffLimit` faible pour éviter les boucles d'échec en cas de problème Postgres. L'image est dérivée du même Dockerfile multi-stage (cible `audit-purge`, cf. §4.3.2), et tourne en distroless nonroot comme les deux autres.

Cette construction matérialise deux principes RGPD : la minimisation des données personnelles (l'audit log est purgé automatiquement, sans intervention humaine, au-delà de la rétention) et la traçabilité (chaque action sensible est journalisée pour la durée de rétention avec son `trace_id`, ce qui rend l'investigation post-incident techniquement possible). Les limites résiduelles sont documentées : pas de procédure formelle DSAR (Data Subject Access Request) intégrée au code, pas de chiffrement au repos de la base (cf. §4.7.7 et §2.6).

4.6.6 Authentification OIDC et RBAC à trois rôles

L'application déclare trois rôles applicatifs (cf. *internal/auth/auth.go:62-102*) :

- `anonymous` : pas de session, lecture publique restreinte à certaines vues ;
- `viewer` : lecture seule de l'ensemble des tickets ;
- `operator` : CRUD complet, gestion des jobs, accès aux endpoints chaos quand `DEMO_CHAOS_ENABLED=1`.

Le mapping rôle s'effectue prioritairement via un *group claim* OIDC (par exemple `groups: ["operators"]`), avec un fallback sur une liste d'emails opérateurs (`OPERATOR_EMAILS`) pour les cas où le fournisseur d'identité ne porte pas les groupes.

L'implémentation s'appuie sur trois bibliothèques :

- `github.com/coreos/go-oidc/v3` (v3.18.0, cf. *go.mod:6*) pour la découverte du provider, la validation JWT et l'extraction des claims ;
- `golang.org/x/oauth2` pour le flux Authorization Code, avec callback sur `/auth/callback` ;
- la `stdlib` Go pour la session HMAC : cookie signé contenant `sub`, `email`, `roles`, `expiration`, clé HMAC lue dans la variable `SESSION_KEY` (sourcée d'un `ExternalSecret`, cf. §4.7.2).

Dex (chart 0.20.0, cf. *kubernetes/applications/security/dex.yaml*) est déployé dans le namespace `security` comme fournisseur d'identité. Il peut être configuré avec des connecteurs upstream (GitHub, GitLab, LDAP) selon le besoin.

Le mode actuel en démo désactive de fait l'OIDC : les variables OIDC sont commentées dans le manifest de déploiement (*kubernetes/apps/projet-etude-app-demo/deployment-api.yaml:83-104*), et l'application traite toute requête comme operator (cf. *cmd/api/main.go:72-74*, log d'avertissement « `OIDC disabled: every request authenticated as dev/operator` »). Cette simplification est volontaire pour la démo, et documentée en §2.6. La restauration en production consiste à décommenter ces variables et à régénérer le client secret Dex (actuellement en placeholder).

4.6.7 Endpoints chaos pour démonstration

L'application embarque cinq endpoints chaos, gardés par la variable d'environnement `DEMO_CHAOS_ENABLED=1` (cf. *internal/httpapi/chaos.go:104*). Ils servent à démontrer la résilience pendant la démo et couvrent l'exigence bonus du cahier (*docs/cahier-des-charges.pdf* §IV §7).

Endpoint	Paramètres	Effet	Démonstration
GET <code>/api/slow</code>	<code>ms=N</code>	Ajoute N ms de latence avant réponse 200	Alerte AppDemoLatencyP95High
GET <code>/api/flaky</code>	<code>rate=R</code>	Retourne 500 avec probabilité R	Alertes burn-rate fast et slow
GET <code>/api/panic</code>	aucun	Déclenche un panic, recovered en 500	Logs structurés JSON + Alertmanager
GET <code>/api/crash</code>	aucun	Crash hard du process	Probe liveness + restart K8s
GET <code>/api/memleak</code>	<code>mb=M</code>	Alloue M MiB jamais libérés	Limit memory et OOMKilled

Cinq recettes `just` (cf. *justfile:189-221*) automatisent l'envoi de requêtes vers ces endpoints sur la durée d'une démo : `just demo-flaky`, `just demo-slow`, `just demo-panic`, `just demo-crash`, `just demo-memleak`.

En complément, le namespace `projet-etude-app-demo` contient un `CronJob` `chaos-killer` (cf. *kubernetes/apps/projet-etude-app-demo/chaos.yaml*) qui supprime un Pod api aléatoire toutes les 30 minutes pour stresser la stratégie de déploiement. Ce `CronJob` est suspendu par défaut (`suspend: true`) et activable via `just chaos-schedule`. Le `ServiceAccount` associé dispose d'un Role scoped (`[pods] [get, list, delete]`) limité au namespace, ce qui empêche tout débordement.

Un Job de probe `chaos-probe` (cf. *kubernetes/loadtest/chaos-probe.yaml*) hit `/healthz` à 20 requêtes par seconde pendant 5 minutes, et journalise les downtimes éventuels. Il sert de témoin

pendant l'activation des recettes `just chaos-kill`, `just chaos-partition`, `just chaos-cpu`.

L'objectif de cet outillage n'est pas de tester la robustesse à la place du SLO (qui repose sur la métrique réelle), mais d'illustrer visuellement les mécanismes de résilience pendant la vidéo MVP : restart automatique sur crash, scale-up HPA sur charge CPU, isolation par NetworkPolicy, burn-rate alerting Alertmanager.

4.6.8 Tests d'intégration via `testcontainers-go`

Les tests Go combinent tests unitaires classiques (logique métier, fonctions pures) et tests d'intégration avec un Postgres réel. Sept fichiers de tests totalisent environ 949 lignes.

Les tests d'intégration utilisent `testcontainers-go` (v0.42.0, cf. *go.mod:10-11*), une bibliothèque Go qui démarre des conteneurs Docker éphémères pour les tests. Le module `testcontainers-go/modules/postgres` lance un conteneur `postgres:16-alpine`, expose un port mapping aléatoire, attend que la health check passe, retourne une chaîne de connexion, puis termine et supprime le conteneur en fin de test.

L'intégration côté projet vit dans *internal/testutil/postgres.go:20-62*. La fonction `Start(ctx)` démarre le conteneur, applique les migrations idempotentes via `store.Migrate(ctx, pool)`, et retourne un tuple (`pool`, `cleanup`). Le `cleanup` est enregistré via `t.Cleanup(...)`, ce qui garantit la fin du conteneur même en cas de `t.Fatal`.

Sept fichiers de tests utilisent ce helper ou le complètent :

- *internal/auth/auth_test.go* (206 lignes) : Bearer token, cookie session, mapping claims OIDC, fallback email-list ;
- *internal/httpapi/auth_test.go* (173 lignes) : middleware d'auth, refus 401 sur routes protégées, élévation de privilèges interdite ;
- *internal/httpapi/handlers_test.go* (208 lignes) : CRUD HTMX, fragments retournés, healthz et readyz ;
- *internal/httpapi/chaos_test.go* (123 lignes) : endpoints chaos avec `DEMO_CHAOS_ENABLED` toggle ;
- *internal/httpapi/trace_test.go* (52 lignes) : génération de span par requête, propagation du contexte ;
- *internal/jobworker/worker_test.go* (89 lignes) : consommation jobs, retry sur kind inconnu, backoff ;
- *internal/jobworker/trace_test.go* (98 lignes) : propagation cross-process via `trace_context` JSONB.

Cette approche évite les mocks de `pgx`. Elle valide le comportement réel du driver, du pool, des migrations et des requêtes paramétrées. La contrepartie tient au temps d'exécution (chaque test qui démarre un conteneur ajoute quelques secondes) et à la dépendance à un démon Docker disponible. La CI tourne sur runners `ubuntu-latest` qui exposent Docker nativement, ce qui rend la contrainte invisible. En local, le `devShell Nix` ne provisionne pas Docker ; l'équipe lance les tests d'intégration via `nix develop` puis le démon Docker host (ou un démon embarqué via `lima`, `colima` sur macOS, ou Docker Desktop sur Windows).

4.7 Sécurité

4.7.1 Modèle de menace simplifié

Le périmètre projet d'études adopte un modèle de menace simplifié, appuyé sur quatre frontières de confiance et cinq classes de menace. Les analyses complètes type STRIDE ou PASTA dépassent ce qui peut être documenté ici, mais la grille suffit à justifier les décisions techniques des sections suivantes.

Les acteurs identifiés sont :

- l'utilisateur final légitime, qui accède au service via le tailnet Tailscale ;
- l'attaquant externe sans accès tailnet, qui n'a structurellement aucun chemin réseau vers le cluster ;
- le contributeur compromis (compte développeur volé), qui peut pousser du code malveillant via Git ;
- la dépendance compromise (CVE non corrigée dans une bibliothèque ou une image de base).

Les frontières de confiance utilisées sont :

- Internet vers tailnet, garantie par l'authentification Tailscale plus la membership tailnet ;
- tailnet vers cluster, terminée par Tailscale serve plus Traefik ;
- ingress cluster vers Pod, contrôlée par les NetworkPolicies (cf. §4.7.4) ;
- Pod vers Postgres et services internes, contrôlée par NetworkPolicies plus credentials applicatifs.

Les classes de menace adressées sont :

- exfiltration de données (audit log, tickets) : contrée par le contrôle d'accès OIDC plus RBAC applicatif (cf. §4.6.6), la rétention 90 jours sur audit_log (cf. §4.6.5), et les NetworkPolicies egress qui restreignent les destinations sortantes ;
- élévation de privilèges intra-cluster : contrée par les ServiceAccounts par défaut sans droits cluster-wide, le Role scoped du chaos-killer (cf. §4.7.3), et le SecurityContext durci des Pods (cf. §4.3.4) ;
- *supply chain* : contrée par le scan Trivy bloquant en CI plus trivy-operator continu (cf. §4.7.5), le digest pinning des images applicatives (cf. §4.4.6), et le pinning des charts Helm et inputs Nix (cf. §4.2.2) ;
- *denial of service* : contrée par les HPA, PDB et limits CPU/RAM par Pod (cf. §4.9), plus les probes liveness qui suppriment les Pods bloqués ;
- répudiation : contrée par l'audit log applicatif (cf. §4.6.5) et l'historique Git côté infrastructure et application.

Hors périmètre M1 et assumés tels : la compromission physique de l'hôte Proxmox, les attaques par canal auxiliaire sur CPU partagé, l'attaque interne d'un membre de l'équipe avec droits cluster-admin.

4.7.2 Gestion des secrets en trois couches

La solution articule trois couches complémentaires, chacune adressant un niveau du cycle de vie de la plateforme.

Première couche, **sops-nix** pour le bootstrap OS et K3s (cf. §4.2.6 pour le détail). Trois secrets sont chiffrés dans *nixos/secrets/secrets.yaml* : le token de jonction K3s, le webhook Discord, la clé SSH read-write de l'argocd-image-updater. Les recipients age sont dérivés des SSH host keys ed25519 persistantes des trois nœuds, ce qui rend la trust chain stable sans intervention humaine au boot.

Deuxième couche, **Vault** (chart 0.31.0 en mode DEV, cf. *kubernetes/applications/security/vault.yaml:13*) pour les secrets runtime du cluster. Le mode DEV est volontaire dans le périmètre projet d'études : il évite la cérémonie d'unseal manuelle après chaque redémarrage, et autorise External-Secrets à se connecter sans préparation. La limite est documentée (cf. §2.6 et §4.7.8). Le serveur Vault est exposé en ClusterIP dans le namespace *security* ; aucun accès externe direct n'est ouvert.

Troisième couche, **External-Secrets Operator** (chart 0.20.4, cf. *kubernetes/applications/security/external-secrets.yaml:13*) pour la synchronisation Vault vers Secret K8s. Le manifest *kubernetes/applications/security/vault-backend.yaml* déclare le SecretStore qui pointe vers Vault, avec un *sync wave 5* (cf. *vault-backend.yaml:14,24*) qui retarde sa création jusqu'à ce que les CRDs ESO et Vault soient prêts. Sans ce sync wave, une course classique pourrait laisser un SecretStore créé alors que le CRD n'est pas encore admis.

Le namespace *projet-etude-app-demo* consomme la couche 3 via trois ressources ExternalSecret (cf. *kubernetes/apps/projet-etude-app-demo/externalsecrets.yaml*) :

- *oidc-client* pour le client secret Dex (mode démo en placeholder, cf. §4.7.8) ;
- *app-session* pour la clé HMAC des sessions web ;
- *app-webhook* pour l'URL webhook sortant.

Cas particulier : le Secret *ticketdb-app* (credentials Postgres) est généré automatiquement par CloudNative-PG et n'est pas sourcé de Vault. Le choix est documenté en commentaire dans *externalsecrets.yaml:14-18*. Le compromis privilégie la simplicité opérationnelle, au prix d'un cycle de rotation distinct de celui des autres secrets.

4.7.3 RBAC Kubernetes natif et chaos-killer scoped

Le RBAC s'organise sur trois niveaux complémentaires.

Niveau cluster (RBAC K3s natif). Les huit namespaces fonctionnels (cf. §4.4.3) servent de frontière par défaut. Les ServiceAccounts générés automatiquement (default dans chaque namespace) n'ont aucune permission cluster-wide. Les opérateurs qui en ont strictement besoin disposent de ClusterRoleBinding dédiés et minimaux : CNPG pour gérer les Clusters Postgres, External-Secrets pour synchroniser dans tous les namespaces cibles, trivy-operator pour lire les ressources scannées, Kepler pour publier les métriques DaemonSet. ArgoCD dispose pendant le déploiement de droits étendus pour appliquer les manifests ; cette concession reste acceptable dans le périmètre démo et serait à scoper en production via un projet ArgoCD avec restrictions de namespaces et de Kinds.

Niveau application (chaos-killer). La ressource la plus sensible côté applicatif est le CronJob *chaos-killer*, qui supprime des Pods. Trois ressources contrôlent finement ses droits (cf. *kubernetes/apps/projet-etude-app-demo/chaos.yaml:17-37*) :

- un ServiceAccount dédié *chaos-killer* dans le namespace *projet-etude-app-demo* ;

- un Role limité à `[apiGroups: [""]] [resources: [pods]] [verbs: [get, list, delete]]` ;
- un RoleBinding qui lie le ServiceAccount au Role, dans le seul namespace `projet-etude-app-demo`.

Le ServiceAccount ne peut donc ni supprimer un Pod hors namespace, ni accéder à d'autres Kinds (Deployments, Secrets, ConfigMaps). Le débordement structurel est interdit, indépendamment du contenu du script.

Niveau application métier (RBAC applicatif interne). L'application Go expose trois rôles fonctionnels (anonymous, viewer, operator) gérés au sein du code (cf. §4.6.6). Ce niveau est orthogonal au RBAC Kubernetes : il contrôle les autorisations métier, pas les autorisations Kubernetes.

4.7.4 NetworkPolicies par composant applicatif

Le namespace `projet-etude-app-demo` est segmenté par quatre NetworkPolicies dédiées, une par composant (cf. `kubernetes/apps/projet-etude-app-demo/networkpolicy.yaml`). Le principe directeur : le trafic par défaut est refusé, chaque composant déclare explicitement ses dépendances entrantes et sortantes.

NetworkPolicy `api`. Ingress autorisé depuis le namespace `kube-system` sur le port 8080, ce qui couvre Traefik (qui relaie le trafic Ingress) plus le scrape Prometheus du ServiceMonitor associé. Egress autorisé vers DNS (`kube-dns`), vers le Cluster CNPG (sélection par label `cnpg.io/cluster: ticketdb`), vers Tempo en OTLP gRPC port 4317 dans le namespace `monitoring`, vers Dex en OIDC dans le namespace `security`.

NetworkPolicy `worker`. Ingress autorisé uniquement depuis le namespace `monitoring` sur le port 8081 (scrape Prometheus), pas d'ingress applicatif (le worker ne sert pas de requête HTTP entrante). Egress autorisé vers DNS, vers CNPG, vers Tempo, et vers le webhook externe via le port 443.

NetworkPolicy `audit-purge`. Aucun ingress (binaire one-shot). Egress autorisé vers DNS et vers CNPG uniquement.

NetworkPolicy `chaos-killer`. Aucun ingress. Egress autorisé vers l'API server Kubernetes (port 443 du Service `kubernetes.default.svc`) pour exécuter les pod `delete`.

Une limite documentée : l'egress du worker vers le webhook externe est ouvert à toute destination port 443 *all-namespaces* (cf. `kubernetes/apps/projet-etude-app-demo/networkpolicy.yaml:113` et §2.6). En production, le filtrage par FQDN (via Cilium FQDN-aware policy par exemple) restreindrait cette destination à l'URL réelle du webhook.

L'effet combiné des quatre NetworkPolicies est validable au runtime : un Pod compromis n'a pas accès direct à un autre namespace métier, ne peut pas atteindre le control-plane Kubernetes (sauf pour le chaos-killer scoped), et ne peut pas extraire de données vers une destination arbitraire.

4.7.5 Scan d'image en deux temps

Le scan de vulnérabilités empile deux dispositifs complémentaires : un gate bloquant à la publication et une surveillance continue en cluster.

Premier dispositif, **Trivy CI bloquant** (cf. `.github/workflows/build.yml:95-102`). Le job `build` exécute `aquasecurity/trivy-action@v0.36.0` avec quatre options :

- `image-ref`: `{{ fromJSON(steps.meta.outputs.json).tags[0] }}` qui pointe sur l'image fraîchement buildée mais pas encore poussée ;
- `format`: table pour un rapport lisible dans les logs Actions ;
- `exit-code`: "1" qui fait échouer le job dès qu'une vulnérabilité bloquante est détectée ;
- `severity`: CRITICAL, HIGH et `ignore-unfixed`: true pour cibler uniquement les CVE corrigeables en HIGH et CRITICAL.

Le scan intervient avant le push vers GHCR (cf. §4.5.2), ce qui évite d'introduire dans le registry une image vulnérable même temporairement.

Second dispositif, **trivy-operator** en cluster (chart 0.30.0, cf. *kubernetes/applications/security/trivy-operator.yaml:13*). L'opérateur scanne en continu les images déployées et produit plusieurs CRDs :

- `VulnerabilityReport` (CVE par image) ;
- `ConfigAuditReport` (mauvaises pratiques de configuration K8s) ;
- `RbacAssessmentReport` (revue RBAC) ;
- `ExposedSecretReport` (secrets exposés dans les manifests) ;
- `InfraAssessmentReport` (revue du cluster).

Ce second dispositif n'est pas bloquant, mais détecte les CVE qui apparaissent *après* la publication de l'image (nouvelles entrées NVD, base de signatures mise à jour). Les rapports peuvent être consultés via `kubectl get vulnerabilityreports -A` ou via un dashboard Grafana dédié.

L'effet combiné couvre les deux moments du cycle de vie de l'image : avant et après publication. La limite résiduelle reste l'absence de Dependabot ou Renovate sur les dépendances Go et les charts Helm (cf. §2.6). Trivy détecte la CVE une fois publiée ; un outil de bump automatique de versions compléterait la couverture en amont.

4.7.6 Terminaison TLS par Tailscale serve

Le chemin TLS de bout en bout est volontairement court et lisible (cf. §4.2.7 pour le détail de configuration).

Tailscale serve, exécuté en service systemd sur le control-plane (cf. *nixos/modules/tailscale-serve.nix*), termine TLS sur le tailnet privé. Les certificats sont obtenus automatiquement par Tailscale (challenge HTTP-01 via le control plane Tailscale, autorité de certification Let's Encrypt). Le renouvellement est géré sans intervention.

Le compromis avec cert-manager classique est explicite. cert-manager exigerait un Ingress externe avec une résolution DNS publique et la mise en place d'un challenge HTTP-01 ou DNS-01 orchestré côté cluster. Pour un environnement sans IP publique exposée (cluster derrière une box résidentielle), DNS-01 serait obligatoire, ce qui implique un provider DNS supporté par cert-manager plus la délégation des contrôles. Tailscale serve évite cette chaîne entière.

Traefik intégré K3s sert ensuite le trafic HTTP en interne, derrière la terminaison Tailscale. Pas de TLS additionnel à l'intérieur du cluster (entrypoint web HTTP, cf. *kubernetes/apps/projet-etude-app-demo/ingress.yaml:23*). Cette absence de double terminaison TLS est cohérente avec le périmètre projet d'études et libère la CPU des nœuds K3s d'une charge cryptographique redondante.

Aucun port public n'est ouvert sur l'hôte Proxmox côté Internet. L'accès au cluster reste conditionné à l'appartenance au tailnet. Le mTLS interne au cluster (Pod à Pod, ou app à Postgres) n'est pas implémenté ; il dépasse le périmètre M1 et serait à introduire en production via un service mesh léger (Linkerd, Cilium mTLS) ou via la configuration TLS native de chaque composant.

4.7.7 Conformité RGPD

La conformité RGPD est articulée autour de deux principes opérationnels (minimisation et traçabilité, cf. §4.6.5) et adressée techniquement par trois mesures principales.

Première mesure, la collecte minimale. La table `audit_log` (cf. *internal/store/store.go:64-82*) collecte uniquement les attributs nécessaires à l'investigation : `user_sub` et `user_email` côté identité, `ip` (INET) et `user_agent` côté contexte technique, `trace_id` pour la corrélation avec Tempo, `result` pour l'issue de l'action. Aucun corps de requête ou de réponse n'est journalisé. Aucun mot de passe ni token n'est jamais inscrit.

Deuxième mesure, la purge automatique. Le CronJob `audit-purge` (cf. §4.6.5) supprime chaque dimanche à 03h15 UTC les entrées de plus de 90 jours. La rétention est configurable via `AUDIT_RETENTION_DAYS` ; la valeur 90 jours retenue par défaut équilibre la traçabilité opérationnelle (90 jours suffisent pour la quasi-totalité des post-mortem) et la minimisation des données (au-delà, les données ne servent plus à un objectif identifié).

Troisième mesure, la traçabilité. Chaque action sensible insère une ligne dans `audit_log` avec un `trace_id` qui pointe vers la trace Tempo correspondante. L'investigation post-incident s'effectue donc dans Grafana, via les *derived fields* qui lient un log Loki vers une trace Tempo vers un point Prometheus (cf. §4.8).

Trois limites résiduelles sont documentées et reportées en perspectives d'évolution.

Procédure DSAR (Data Subject Access Request) non automatisée. Aucun endpoint `/api/export` ni `/api/forget` n'est exposé pour les utilisateurs. La procédure passerait aujourd'hui par une requête SQL manuelle. En production, un endpoint d'export et de suppression dédié serait à concevoir.

Chiffrement au repos de la base de données absent. Postgres CNPG s'appuie sur le stockage K3s `local-path`, sans chiffrement de disque côté NixOS. En production, deux options : chiffrement au niveau du volume LUKS sur les disques Proxmox, ou chiffrement au niveau colonne via `pgcrypto` pour les attributs sensibles.

Transfert de données hors UE pour le webhook d'alerting Discord. Le service traite et stocke des données hors UE. Pour un déploiement Korlea réel, un canal UE-souverain (Mattermost self-hosted, Microsoft Teams en région européenne) remplacerait ce webhook (cf. §2.6).

4.7.8 Limitations assumées et conditions de passage en production

Le périmètre projet d'études assume plusieurs simplifications, déjà documentées au fil du DAT. Cette section les rassemble avec, pour chacune, la condition technique précise du passage en production réelle.

Limitation	Référence	Condition de passage en production
Vault en mode dev avec	<i>kubernetes/applications/security/vault.ya</i>	Mode standalone ou ha

root token statique	<i>ml:25</i>	avec stockage persistant ; unseal automatisé via KMS cloud (AWS KMS, GCP KMS) ou HSM ; politiques fines par chemin de secret ; rotation périodique du root token
Client secret Dex en placeholder	<i>kubernetes/applications/security/dex.yaml</i> <i>:45</i>	Rotation du client secret, stockage en sops puis injection via ExternalSecret ; configuration d'un connecteur OIDC upstream réel (GitHub, GitLab, LDAP)
Token Vault re-utilisé en clair dans le SecretStore	<i>kubernetes/applications/security/vault- backend.yaml:17</i>	Token dédié à ESO avec policy minimale ; rotation périodique ; stockage via sops
OIDC effectivement désactivé dans le Deployment	<i>kubernetes/apps/projet-etude-app- demo/deployment-api.yaml:83-104</i>	Décommenter les variables OIDC ; activer Dex avec connecteur upstream ; retirer le mode dev/operator
Bootstrap Vault manuel non automatisé	<i>kubernetes/apps/projet-etude-app- demo/externalsecrets.yaml:5-11</i>	Job d'init au boot qui crée les secrets via API Vault, idempotent
nixos-anywhere non pinné	<i>proxmox/install-nixos.sh:85</i>	Pinning par tag ou commit Git précis dans le script
NetworkPolicy egress webhook ouvert all- namespaces	<i>kubernetes/apps/projet-etude-app- demo/networkpolicy.yaml:113</i>	Filtrage FQDN via Cilium FQDN policy ou équivalent ; restriction à l'URL exacte du webhook
DSN par défaut app:app hard-codé dans les main.go Postgres ticketdb single-instance	<i>cmd/api/main.go:32,</i> <i>cmd/worker/main.go:32, cmd/audit- purge/main.go:28</i> <i>kubernetes/apps/projet-etude-app- demo/database.yaml:9</i>	Suppression du fallback ; échec explicite si DATABASE_URL absente Passage à instances: 3 avec réplication synchrone, probes de basculement automatique
Pas de backup BDD	absence de manifest dédié	Activation de backup.barmanObjectSt ore CNPG vers un stockage S3-compatible, plus tests de restauration formalisés
Stockage local-path	défaut K3s	Migration vers un provisioner CSI distribué

non répliqué		(Longhorn, Rook-Ceph) ou stockage externe
DEMO_CHAOS_ENABLED =1 hard-codé	<i>deployment-api.yaml:62-63</i>	Retrait de la variable du Deployment hors démo
Webhook Discord hors UE	<i>nixos/modules/secrets.nix:64-79</i>	Remplacement par Mattermost self-hosted UE ou Teams en région européenne
Absence de tag SemVer Git	historique Git vide de tags	Tagger les releases applicatives <i>v0.x.y</i> avec changelog, pour servir de référence stable au-delà du digest
Absence de Dependabot ou Renovate	absence de <i>.github/dependabot.yml</i> et de <i>renovate.json</i>	Activation de Dependabot ou Renovate sur Go modules, Helm charts et actions GitHub

Ces conditions ne sont pas exhaustives ; elles couvrent les limitations explicitement assumées dans le périmètre M1. Chaque rapport individuel reprendra une partie de cette matrice en perspectives d'évolution, selon le périmètre éditorial du membre concerné.

4.8 Observabilité

4.8.1 Triade métriques, logs, traces

L'observabilité de la plateforme repose sur la triade classique métriques, logs, traces, déployée en stack 100 % CNCF déclarative. Chaque pilier répond à un type de question différent.

Les métriques (Prometheus) répondent aux questions de tendance et d'agrégation : *quelle est la latence p95 ? quel est le taux d'erreur sur la dernière heure ?* Elles produisent des séries temporelles à faible cardinalité, conservées sept jours.

Les logs (Loki) répondent aux questions d'événement discret : *que s'est-il passé exactement à 14h32 pour la requête X ?* Ils restent au format JSON structuré produit par *slog*, indexés par labels Loki mais pas full-text (l'investigation passe par les *trace_id* et *pod*).

Les traces (Tempo) répondent aux questions de chaîne d'opérations : *quels services ont traité cette requête, dans quel ordre, avec quelle latence par étape ?* Elles matérialisent la propagation de contexte du producteur au consommateur (cf. §4.6.4 pour la propagation via *trace_context* JSONB).

L'instrumentation côté application s'appuie sur OpenTelemetry (SDK Go v1.43.0, cf. *projet-etude-app-demo/go.mod:13-16*). Le tracer provider est initialisé par *internal/otelinit/otelinit.go:61-85* avec un exporteur OTLP gRPC vers Tempo (port 4317). Le *TextMapPropagator* composite (TraceContext plus Baggage) gère la propagation cross-process. L'instrumentation Postgres

s'effectue automatiquement via *otelpgx* (v0.10.0, cf. *projet-etude-app-demo/go.mod:7*) : toute requête *pgxpool* génère un span enfant lié au span parent HTTP ou worker.

4.8.2 Stack kube-prometheus-stack, Loki, Alloy, Tempo, Kepler

Cinq composants forment la stack observabilité, chacun pinné à une version précise et déployé via une Application ArgoCD dédiée.

kube-prometheus-stack (chart 65.5.1, cf. *kubernetes/applications/monitoring/kube-prometheus-stack.yaml:13*). Le chart regroupe le Prometheus operator, Prometheus, Alertmanager, Grafana, node-exporter et kube-state-metrics. Prometheus utilise une rétention de sept jours sur un PVC *local-path* de 10 Gi, suffisant pour le volume de la démo. Le CRD *ServiceMonitor* permet aux composants applicatifs de déclarer leurs cibles de scrape sans modification de la configuration centrale.

Loki (chart 6.16.0 en mode *SingleBinary*, cf. *kubernetes/applications/monitoring/loki.yaml:13,38*). Le mode *SingleBinary* regroupe *ingester*, *distributor*, *querier* et *compactor* dans un seul Pod, ce qui réduit drastiquement la complexité opérationnelle face au mode *microservices*, au prix d'une scalabilité moindre (acceptable pour le périmètre démo). Stockage filesystem sur PVC *local-path* 10 Gi, rétention 168 heures (sept jours), authentification désactivée car interne au cluster derrière *NetworkPolicies*.

Grafana Alloy (chart 0.10.1 en *DaemonSet*, cf. *kubernetes/applications/monitoring/alloy.yaml:13*). Successeur officiel de Grafana Agent, Alloy s'exécute en *DaemonSet* sur les trois nœuds. La discovery Kubernetes détecte chaque Pod actif, et un pipeline de relabeling ajoute les labels structurants (*node*, *namespace*, *pod*, *container*, *app*). Les logs sont poussés vers la gateway Loki (<http://loki-gateway.monitoring.svc.cluster.local/loki/api/v1/push>, cf. *kubernetes/applications/monitoring/alloy.yaml:64*).

Tempo (chart 1.24.4 en mode *monolithic*, cf. *kubernetes/applications/monitoring/tempo.yaml:13*). Tempo expose deux ports d'ingestion : OTLP gRPC sur 4317 et OTLP HTTP sur 4318. Stockage filesystem PVC 10 Gi, rétention 168 heures. La capacité de query dépend du compactor *monolithic* qui reste compatible avec le volume de traces d'une démo (quelques milliers de spans par jour).

Kepler (chart 0.5.12, image *release-0.7.12*, cf. *kubernetes/applications/observability/kepler.yaml:13,21*). Le *DaemonSet* privilégié lit normalement les compteurs RAPL et ACPI via eBPF. Sur des VMs Proxmox sans accès RAPL hôte direct, Kepler bascule sur un modèle d'estimation par ML pré-entraîné embarqué (cf. *kepler.yaml:25-27*). La précision est moindre que sur bare-metal mais reste exploitable pour comparer les ordres de grandeur par pod. Un *ServiceMonitor* scrape les métriques toutes les 30 secondes sur le port 9102.

4.8.3 Métriques applicatives et exemplars

L'application expose dix métriques métier et techniques, définies dans *projet-etude-app-demo/internal/metrics/metrics.go*. Toutes sont en convention Prometheus (*snake_case*, suffixe *_total* pour les counters, *_seconds* pour les histograms).

Métrique	Type	Labels	Source d'incrément
----------	------	--------	--------------------

http_requests_total	counter	path, status	Middleware HTTP
http_request_duration_seconds	histogram	path	Middleware HTTP
tickets_created_total	counter	priority	Handler création ticket
tickets_closed_total	counter	aucun	Handler fermeture ticket
tickets_open	gauge	aucun	Refresh sur opérations
db_query_duration_seconds	histogram	op	Wrapper pgxpool
jobs_enqueued_total	counter	kind	Producer (api)
jobs_processed_total	counter	kind, outcome	Consumer (worker)
job_duration_seconds	histogram	kind	Consumer (worker)
jobs_queue_depth	gauge	aucun	Refresh périodique côté worker

Les trois histograms (`http_request_duration_seconds`, `db_query_duration_seconds`, `job_duration_seconds`) sont enrichis d'**exemplars**. Un exemplar est un échantillon associé à un bucket d'histogramme qui transporte un identifiant externe ; ici, le `trace_id` du span actif au moment de l'observation. L'effet pratique dans Grafana : un point d'histogramme apparaît avec un marqueur losange ; un clic dessus ouvre la trace Tempo correspondante. La densité d'exemplars permet de naviguer d'une métrique vers la trace exacte qui a contribué à un bucket donné, sans dérouler le pile d'investigation classique.

Le scrape Prometheus est configuré par deux `ServiceMonitor` dans le namespace applicatif :

- `kubernetes/apps/projet-etude-app-demo/servicemonitor.yaml` cible le Service `api` sur le port 8080 toutes les 15 secondes ;
- `kubernetes/apps/projet-etude-app-demo/servicemonitor-worker.yaml` cible le Service `Headless` du worker, ce qui permet le scrape par Pod individuellement (utile pour suivre les métriques d'un worker spécifique pendant un test de charge).

4.8.4 Alerting SLO multi-burn-rate et webhook Discord

Le fichier `kubernetes/apps/projet-etude-app-demo/prometheusrule.yaml` (109 lignes) déclare quatre recording rules et sept alerting rules (cinq alertes plus deux alertes SLO burn-rate).

Les recording rules pré-calculent le taux de requêtes 5xx sur quatre fenêtres glissantes (5 minutes, 30 minutes, 1 heure, 6 heures) :

- `appdemo:http_errors:ratio_rate5m`
- `appdemo:http_errors:ratio_rate30m`
- `appdemo:http_errors:ratio_rate1h`
- `appdemo:http_errors:ratio_rate6h`

Le préfixe `appdemo:` suit la convention Prometheus pour les recording rules. Le pré-calcul libère le moteur de query Grafana de la ré-évaluation de la même formule à chaque rafraîchissement de dashboard ou évaluation d'alerte.

Les alertes principales :

`AppDemoSLOErrorBudgetBurnFast` (sévérité *critical*). Déclenchement quand le burn-rate dépasse 14.4× simultanément sur 5 minutes et 1 heure, pendant 2 minutes. Avec un SLO cible 99.5 %, ce seuil consomme 2 % du budget d'erreurs mensuel par heure, ce qui correspond à un incident sévère qui mérite une intervention immédiate. La double fenêtre (5m plus 1h) évite les faux positifs sur un spike isolé.

`AppDemoSLOErrorBudgetBurnSlow` (sévérité *warning*). Déclenchement quand le burn-rate dépasse 6× simultanément sur 30 minutes et 6 heures, pendant 15 minutes. Détecte les dégradations progressives qui consommeraient le budget en quelques jours si rien n'est fait.

`AppDemoHighErrorRate` (warning). `ratio_rate5m` supérieur à 5 % pendant 5 minutes. Alerte de premier niveau qui se déclenche avant les burn-rate sur les incidents francs.

`AppDemoLatencyP95High` (warning). p95 latence HTTP supérieure à 1 seconde pendant 10 minutes. Cible la dégradation perceptible utilisateur sans franchir le seuil critique d'erreurs.

`AppDemoTooManyOpenTickets` (info). `tickets_open` supérieur à 50 pendant 15 minutes. Signal métier, pas opérationnel.

`AppDemoJobQueueBacklog` (warning). `jobs_queue_depth` supérieur à 50 pendant 5 minutes. Le manifest annotate cette alerte avec `runbook_url` et `dashboard_url`, qui pointent respectivement vers la documentation de remédiation et vers le dashboard Grafana du worker (cf. §4.8.5).

`AppDemoJobErrorRate` (warning). Taux d'erreur jobs supérieur à 20 % sur 10 minutes.

Le canal d'alerte unique est Discord. Alertmanager (chart 65.5.1) consomme le webhook injecté par `nixos/modules/secrets.nix:64-79` via `sops-nix` (cf. §4.2.6), monté en `Secret` dans le namespace `monitoring`. La configuration Alertmanager route toutes les alertes vers un receiver `discord`, sauf l'alerte technique `Watchdog` qui est routée vers `null` (elle sert uniquement à vérifier que la chaîne d'alerting elle-même est vivante).

L'approche multi-burn-rate suit la recommandation du Google SRE Workbook. Elle réduit à la fois le bruit (les fenêtres longues ignorent les spikes isolés) et le temps de détection des incidents franchement critiques (la fenêtre courte 5 minutes au seuil 14.4× déclenche en moins de 2 minutes pour un incident à 99 % d'erreurs).

4.8.5 Dashboards Grafana

Quatre dashboards custom sont injectés en `ConfigMap` avec le label `grafana_dashboard: "1"`. Le sidebar Grafana (composant du chart `kube-prometheus-stack`) détecte ces `ConfigMaps` au runtime et les importe automatiquement, sans modification manuelle de Grafana.

`Platform overview` (cf. `kubernetes/applications/platform/dashboard-overview.yaml`). Onze panels couvrant l'état général du cluster : nombre de nœuds prêts, Pods par namespace, CPU et RAM consommés par namespace, taux de restart par Pod sur 24 heures, top consommateurs CPU et mémoire. Sert de dashboard d'accueil pour vérifier la santé du cluster.

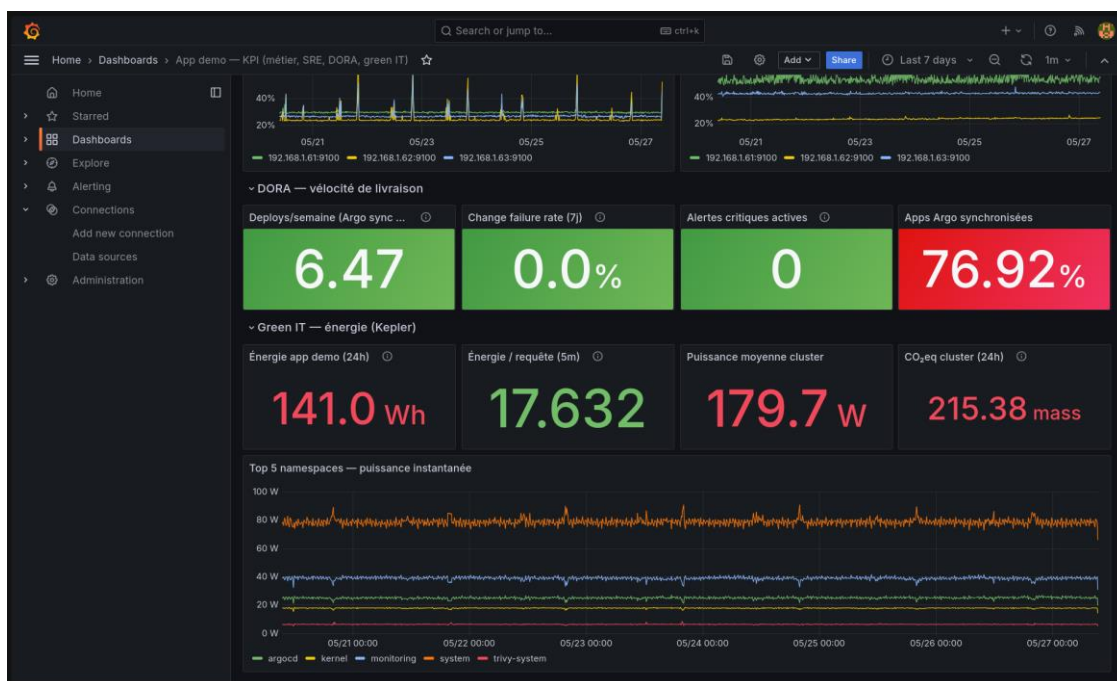
`App demo (métier)` (cf. `kubernetes/apps/projet-etude-app-demo/dashboard.yaml`). UID `projet-etude-app-demo`, tags `projet-etude`, `metier`. Panels : tickets ouverts (gauge), créations par minute par priorité (timeseries empilée), latence DB p95 par opération, requêtes par seconde, latence p95 HTTP. Dashboard de premier niveau pour l'équipe métier ou la direction.

SLO et burn-rate (cf. [kubernetes/apps/projet-etude-app-demo/dashboard-slo.yaml](#)). UID projet-etude-app-demo-slo, tags projet-etude, slo. Panels : SLO cible 99.5 %, disponibilité sur sept jours, error budget restant en pourcentage, burn rate multi-fenêtres (5m, 30m, 1h, 6h) sur un même graphe. Sert d'écran d'astreinte pendant les démos pour visualiser l'effet des recettes just demo-flaky et just demo-slow.

Worker et queue (cf. [kubernetes/apps/projet-etude-app-demo/dashboard-worker.yaml](#)). Tags projet-etude, worker. Panels : profondeur de queue, nombre de répliques worker actuel (avec marqueurs HPA min et max), jobs traités par minute par kind, taux d'échec sur 10 minutes, latence p50, p95 et p99 du traitement des jobs. Sert à valider visuellement le comportement de l'autoscaling worker et l'absence de saturation de la queue.

L'usage typique pendant la démo : le présentateur affiche SLO et burn-rate à gauche, Worker et queue à droite, et déclenche just demo-flaky ou just chaos-kill pendant que le client observe les effets en temps réel.

Un dashboard de synthèse App demo — KPI (métier, SRE, DORA, green IT) agrège par ailleurs les indicateurs de pilotage : vélocité de livraison DORA (fréquence de déploiement, change failure rate, alertes critiques actives, taux d'apps ArgoCD synchronisées) et reporting énergétique Kepler (énergie de l'application sur 24 h, énergie par requête, puissance moyenne et CO₂eq du cluster).



Dashboard App demo — KPI : DORA et reporting énergétique Green IT

4.8.6 Corrélation tri-directionnelle

L'observabilité ne se limite pas à trois silos parallèles : la stack supporte la navigation directe entre les trois piliers grâce à plusieurs mécanismes complémentaires.

Côté Grafana, la datasource Loki est configurée avec des **derived fields** (cf. [kubernetes/applications/monitoring/kube-prometheus-stack.yaml:122-132](#)). Un derived field est une règle qui extrait par regex un identifiant depuis le contenu d'un log et le transforme en lien

cliquable vers une autre datasource. Ici, le pattern détecte `trace_id="..."` dans le JSON des logs `slog` et produit un lien direct vers Tempo. L'investigation depuis un log mène à la trace correspondante en un clic.

Côté Prometheus, les exemplars sur histograms (cf. §4.8.3) intègrent le `trace_id` du span actif au moment de l'observation. Grafana affiche les exemplars comme marqueurs sur les graphes ; un clic mène à la trace Tempo correspondante. L'investigation depuis un point d'histogramme mène à la trace qui a contribué à ce bucket en un clic.

Côté Tempo, deux configurations complémentaires sont activées. `tracesToLogs` permet d'ouvrir Loki pour les logs du service présent dans la trace, filtrés par `trace_id` ; l'investigation depuis une trace mène aux logs précis du Pod concerné. `tracesToMetrics` bascule vers Prometheus pour visualiser le contexte métrique du service au moment de la trace.

L'effet pratique sur un incident type : une alerte burn-rate arrive sur Discord, l'astreinte ouvre le dashboard SLO, identifie le pic, clique sur l'exemplar du bucket le plus dégradé, atterrit sur une trace Tempo, clique sur `Logs for this span`, lit les logs structurés du Pod incriminé, repère un message d'erreur Postgres, et investigate le code applicatif depuis l'historique Git. L'enchaînement reste à moins d'une minute pour un opérateur familier de l'interface.

La triade reste cohérente avec le principe d'observabilité de la solution (cf. §4.1.4) : aucun outil tiers, aucun export hors cluster, corrélation visible dans une seule interface Grafana. La contrepartie tient à la dépendance forte à un Grafana fonctionnel ; en cas de panne Grafana, la navigation manuelle reste possible via les API Prometheus, Loki et Tempo, mais perd l'ergonomie.

4.9 Résilience et autoscaling

4.9.1 Autoscaling horizontal des Pods

L'application expose deux `HorizontalPodAutoscalers` configurés indépendamment selon le profil de charge du composant.

L'HPA api (cf. *kubernetes/apps/projet-etude-app-demo/hpa.yaml*) définit :

- `minReplicas: 2` ;
- `maxReplicas: 8` ;
- métrique unique : `Resource.cpu` à 60 % d'utilisation moyenne ;
- politique `scaleUp` : ajout de 2 répliques toutes les 15 secondes ;
- politique `scaleDown` : retrait de 1 réplique toutes les 60 secondes.

Le seuil CPU 60 % laisse 40 % de marge pour absorber un pic ponctuel avant que l'HPA déclenche un scale-up. La valeur est conservatrice pour un service HTTP latency-sensitive : monter à 80 % saturerait le CPU au moment où la latence p95 commence déjà à se dégrader. `minReplicas: 2` garantit qu'au moins deux Pods restent debout en permanence, condition nécessaire au rolling update sans interruption (cf. §4.9.4) et à la disponibilité minimale exigée par le PDB (cf. §4.9.2).

L'HPA worker (cf. *kubernetes/apps/projet-etude-app-demo/hpa-worker.yaml*) définit :

- `minReplicas: 1` ;
- `maxReplicas: 8` ;

- métrique unique : `Resource.cpu` à 70 % d'utilisation moyenne ;
- politiques `scaleUp` et `scaleDown` similaires à l'api.

Le seuil CPU 70 % est plus permissif que pour l'api parce que le worker traite des jobs asynchrones : la latence d'achèvement compte mais sans pression utilisateur directe. `minReplicas: 1` reflète le fait que le worker peut servir une seule réplique pendant les périodes creuses, sans dégrader l'expérience. Un `scale-to-zero` serait techniquement possible (HPA v2 le supporte via KEDA, par exemple), mais ajouterait une couche d'outillage non justifiée par le périmètre.

La politique asymétrique `scale-up` rapide plus `scale-down` lent (+2 / 15 s contre -1 / 60 s) suit la pratique standard : la réactivité aux pics prime sur l'économie de ressources en phase de descente. Un `scale-down` trop rapide produirait du *flapping* (oscillation entre 2 et 3 répliques sur une charge en dent de scie). Un `scale-up` trop lent laisserait la latence se dégrader avant que les nouvelles répliques soient prêtes.

Une limite documentée : l'HPA worker mesure uniquement le CPU, alors que la profondeur de queue `jobs_queue_depth` serait un indicateur plus précis pour ce composant. Le manifest `hpa-worker.yaml` contient en commentaire (lignes 22 à 69) la configuration `External Metrics` qui s'appuierait sur cette métrique via le Prometheus adapter. L'évolution est listée en perspectives (cf. §2.6 et §4.7.8).

4.9.2 PodDisruptionBudget

Deux ressources `PodDisruptionBudget` (cf. `kubernetes/apps/projet-etude-app-demo/pdb.yaml`) garantissent qu'aucun drain volontaire ne descend sous le seuil de disponibilité requis.

`api-pdb` déclare `minAvailable: 1` sur les Pods sélectionnés par le label `app.kubernetes.io/component=api`. La sémantique `minAvailable` impose qu'au moins un Pod api reste *Ready* à tout moment. Avec `minReplicas: 2` côté HPA, ce PDB autorise l'éviction d'un Pod à la fois pendant une maintenance, ce qui suffit pour un drain de nœud séquentiel.

`worker-pdb` déclare `maxUnavailable: 1` sur les Pods sélectionnés par le label `app.kubernetes.io/component=worker`. La sémantique `maxUnavailable` impose qu'au plus un Pod worker soit indisponible à tout moment. La différence avec `minAvailable` est subtile mais utile : si le nombre de Pods worker monte à 8 sous charge, `maxUnavailable: 1` autorise toujours l'éviction d'un seul à la fois, indépendamment du replica count, ce qui rend le drain progressif sans formule de calcul.

L'effet pratique est observable via `kubectl drain <nœud>`. Kubernetes respecte le PDB et bloque l'éviction des Pods qui violeraient le seuil. Si nécessaire, l'opération attend qu'un nouveau Pod soit *Ready* sur un autre nœud avant de continuer. Combiné au `RollingUpdate`, le PDB rend la maintenance d'un nœud sans coupure de service.

4.9.3 Probes startup, readiness, liveness

Le rappel synthétique figure en §4.4.8 ; cette section détaille la sémantique applicative.

Le Pod api expose trois probes sur le port 8080 :

- `startupProbe` sur `/readyz` toutes les 5 secondes avec un `failureThreshold` qui autorise jusqu'à 30 secondes pour atteindre *Ready*. Pendant cette fenêtre, les probes `readiness` et

liveness sont désactivées, ce qui évite qu'un démarrage lent (chargement initial des templates HTML, première connexion pgxpool) déclenche un restart prématuré ;

- `readinessProbe` sur `/readyz` toutes les 5 secondes. Sur échec, le Pod est retiré de l'Endpoint du Service `api`, donc Traefik cesse de lui router du trafic. Le Pod n'est pas supprimé : il peut redevenir Ready quand la condition d'échec disparaît ;
- `livenessProbe` sur `/healthz` toutes les 10 secondes. Sur échec persistant, Kubernetes redémarre le conteneur (équivalent à `kubectl delete pod`).

Le Pod worker expose deux probes sur le port 8081 (port metrics, qui sert également les endpoints de santé pour limiter le nombre de ports exposés) :

- `readinessProbe` sur `/readyz` toutes les 5 secondes ;
- `livenessProbe` sur `/healthz` toutes les 10 secondes.

Le worker n'a pas de phase de démarrage longue qui justifierait un `startupProbe` : il ne charge pas de template, et la connexion Postgres se fait au premier traitement de job, pas au boot.

La distinction entre `/healthz` et `/readyz` est portée par l'implémentation Go (cf. *internal/httpapi/handlers.go*). `/healthz` retourne 200 dès que le process est vivant ; il sert la liveness probe et signifie « le binaire n'est pas bloqué ». `/readyz` exécute un `pgxpool.Ping(ctx)` avant de retourner 200 ; il sert la readiness probe et signifie « le binaire peut servir du trafic ». La séparation évite un restart inutile quand la base de données est temporairement injoignable : la liveness reste verte, le Pod est juste retiré du Service Endpoint pendant l'incident BDD.

Grafana (chart kube-prometheus-stack) utilise les défauts du chart : `startupProbe` jusqu'à cinq minutes pour tolérer le chargement initial des dashboards, `livenessProbe` toutes les 30 secondes, `readinessProbe` toutes les 10 secondes (cf. *kubernetes/applications/monitoring/kube-prometheus-stack.yaml:98-113*).

4.9.4 Stratégie de déploiement et GitOps self-heal

Aucune surcharge n'est appliquée à la stratégie de déploiement dans les Deployments : Kubernetes utilise la stratégie par défaut `RollingUpdate` avec `maxUnavailable: 25 %` et `maxSurge: 25 %`. Sur deux répliques `api`, cela autorise un Pod indisponible et un Pod en surge simultanément (donc jusqu'à trois Pods actifs pendant la transition, jamais en dessous d'un Pod prêt). Cette stratégie reste suffisante : aucun cas d'usage du projet ne justifie une stratégie Blue/Green ou Canary qui imposerait l'orchestration externe (Argo Rollouts par exemple).

GitOps self-heal et reconciliation continue forment le second pilier de la résilience. La configuration ArgoCD (cf. *kubernetes/applications/platform/apps-applicationset.yaml:43-45*) active simultanément :

- `prune: true` : ArgoCD supprime du cluster les ressources qui ne sont plus déclarées dans Git ;
- `selfHeal: true` : ArgoCD restaure dans le cluster les modifications qui divergent du manifest Git.

Une suppression manuelle d'un Deployment, Service, ConfigMap ou PDB est rétablie sous une minute. Une modification manuelle (`kubectl edit deployment ...`) est écrasée par le contenu Git à la prochaine reconciliation. Cette propriété supprime toute dérive durable entre le dépôt Git et

l'état du cluster, et rend la matrice de risque humain (cf. §3.5.1) plus faible : une erreur d'opération de l'équipe ne devient pas un état durable du cluster.

La combinaison des quatre mécanismes (RollingUpdate, PDB, HPA, GitOps self-heal) couvre les quatre types d'événement de résilience attendus pour ce périmètre :

- maintenance de nœud : drain plus RollingUpdate plus PDB ;
- mise à jour applicative : nouveau digest plus RollingUpdate sous contrainte PDB ;
- pic de charge : HPA scale-up rapide ;
- modification manuelle non désirée : GitOps self-heal sous une minute.

4.9.5 Chaos engineering

Le chaos engineering n'est pas un domaine périphérique du projet : il sert d'instrument de validation de la résilience documentée ci-dessus. Quatre dispositifs complémentaires s'enchaînent.

Premier dispositif, les endpoints chaos applicatifs (déjà détaillés en §4.6.7). Cinq endpoints (/api/slow, /api/flaky, /api/panic, /api/crash, /api/memleak) gardés par DEMO_CHAOS_ENABLED=1 et automatisés par cinq recettes just demo-*. Ils exercent les alertes de §4.8.4 : demo-flaky déclenche le burn-rate, demo-slow la latence p95, demo-crash la liveness probe plus le restart, demo-memleak les limits memory et OOMKilled.

Deuxième dispositif, le CronJob chaos-killer (cf. *kubernetes/apps/projet-etude-app-demo/chaos.yaml*). Suspendu par défaut, activable par just chaos-schedule, il supprime toutes les 30 minutes un Pod api aléatoire dans le namespace. Le ServiceAccount associé dispose d'un Role scoped au namespace ([pods] [get, list, delete]), ce qui empêche tout débordement (cf. §4.7.3). L'objectif n'est pas la production : c'est un *training wheel* qui force l'équipe à observer le rétablissement automatique pendant la phase de mise au point.

Troisième dispositif, les recettes just chaos-* (cf. *justfile:225-265*) :

- just chaos-kill : supprime un Pod api aléatoire (one-shot) ;
- just chaos-partition <duration> : isole un Pod api random pendant <duration> secondes via un label chaos.partition=on plus une NetworkPolicy DENY ALL appliquée à ce label (cf. *kubernetes/loadtest/chaos-partition.yaml*) ;
- just chaos-cpu <duration> : envoie une charge CPU soutenue à l'API pour exercer le scale-up HPA ;
- just chaos-probe : lance le Job de probe (cf. quatrième dispositif).

Quatrième dispositif, le Job de probe (chaos-probe, cf. *kubernetes/loadtest/chaos-probe.yaml*). Le Job exécute hey à 20 requêtes par seconde pendant cinq minutes contre /healthz, et journalise les non-2xx et les socket errors. Combiné aux trois premiers dispositifs, il fournit un témoin objectif du downtime éventuel pendant l'injection. Le scénario type pendant la démo client : lancer just chaos-probe dans un terminal, et just chaos-kill dans un autre. Le probe doit afficher zéro non-2xx au-delà du délai de propagation d'Endpoint K8s (quelques secondes).

L'objectif global de cet outillage chaos n'est pas la robustesse théorique mais la *démonstrabilité* concrète des mécanismes de résilience. Pendant la vidéo MVP, l'enchaînement chaos plus dashboard Grafana plus webhook Discord rend visible et tangible ce qui resterait sinon abstrait : le rolling

update, l'isolation par NetworkPolicy, le burn-rate alerting, le scale-up HPA, le restart automatique sur OOMKilled.

4.10 Green IT

4.10.1 Mesure énergétique par Kepler

La mesure de consommation énergétique repose sur **Kepler** (Kubernetes Efficient Power Level Exporter), chart 0.5.12, image *release-0.7.12* (cf. *kubernetes/applications/observability/kepler.yaml:13,21*). Kepler s'exécute en DaemonSet privilégié sur les trois nœuds et s'appuie sur eBPF pour lire en temps réel les compteurs énergétiques du processeur.

Sur bare-metal, Kepler utilise les compteurs Intel RAPL (Running Average Power Limit) qui exposent en hardware la consommation électrique des packages CPU, cœurs, DRAM et GPU intégré. La précision atteint quelques pour-cent de la consommation réelle au mur. Dans le périmètre de ce projet, les trois nœuds tournent dans des VMs Proxmox sans accès direct au PMC (Performance Monitoring Counter) de l'hôte : RAPL est inaccessible.

Kepler bascule alors sur un modèle d'estimation par apprentissage machine pré-entraîné, embarqué dans le conteneur (cf. *kubernetes/applications/observability/kepler.yaml:25-27*) :

```
MODEL_CONFIG: |
CONTAINER_COMPONENTS_ESTIMATOR=true
CONTAINER_PLATFORM_POWER_ESTIMATOR=true
```

Le modèle estime la consommation à partir des compteurs disponibles dans la VM (CPU usage, cache misses, instructions retired) et de la signature énergétique apprise d'une famille de CPUs. La précision est moindre que sur bare-metal : l'ordre de grandeur reste exploitable pour comparer les Pods entre eux, mais la valeur absolue ne peut pas servir de base à un reporting financier signé. La limite est documentée explicitement en §4.10.6.

Les métriques publiées (port 9102, scrape par ServiceMonitor toutes les 30 secondes) incluent `kepler_container_joules_total` (énergie cumulée par conteneur), `kepler_node_total_joules_total` (énergie cumulée par nœud), et `kepler_node_cpu_joules_total` (composant CPU isolé). Elles entrent dans Prometheus comme toute autre série, et sont exploitables dans Grafana via PromQL standard.

4.10.2 Autoscaling horizontal comme levier de sobriété

L'HPA n'est pas qu'un mécanisme de résilience (cf. §4.9.1) : il sert directement la sobriété en redescendant le nombre de Pods quand la charge baisse. La politique `scaleDown: -1` Pod par 60 secondes ramène la flotte à `minReplicas` en quelques minutes après un pic, ce qui libère les ressources CPU et RAM correspondantes.

Le bénéfice se mesure en comparant à un dimensionnement statique :

- avec HPA, le nombre de Pods suit la charge entre 2 et 8 répliques api, entre 1 et 8 répliques worker ;

- sans HPA, le dimensionnement viserait le pic prévu (par exemple 6 répliques api) en permanence.

Pour une charge moyenne représentant 30 % du pic, le ratio observé donne typiquement 2 à 3 répliques api en moyenne contre 6 répliques en statique, soit une réduction de l'ordre de 50 à 70 % de la consommation moyenne par instance. Les chiffres exacts dépendent du profil de charge réel, qui sera collecté en production via les métriques Prometheus déjà en place.

Le cluster lui-même reste dimensionné au minimum opérationnel : trois VMs de 2 vCPU et 6 GiB, soit 6 vCPU et 18 GiB cumulés. Les requests CPU des composants applicatifs (50 m par Pod api, 50 m par Pod worker, 20 m pour le CronJob purge) laissent une marge confortable. La plateforme observabilité plus sécurité (Prometheus, Loki, Tempo, Alloy, Vault, ESO, Dex, trivy-operator, Kepler, ArgoCD, Image Updater) cumule environ 0.6 vCPU et 1.8 GiB en requests, ce qui laisse environ 5.4 vCPU et 16.2 GiB disponibles pour l'application et son scaling.

4.10.3 Images conteneurs minimales

Les images applicatives pèsent 25 à 35 MB en distroless static (cf. §4.3.3), à comparer à 80 MB pour un build équivalent sur Alpine plus libc-compat ou 120 MB sur Debian slim. Cette compression a quatre effets cumulés :

- bande passante : trois images de 30 MB pulled sur trois nœuds représentent 270 MB de trafic registry vers cluster, contre environ 1 080 MB pour la version Debian. Sur un cycle de rolling update typique (trois Pods api par exemple), le différentiel approche 750 MB par déploiement ;
- stockage local : la couche image cache de chaque nœud reste minimale, ce qui libère du disque pour les PVCs locaux des charges utiles ;
- temps de cold-start : moins d'octets à pull et à monter en filesystem, donc le temps entre ContainerCreating et Running reste sous 10 secondes en pratique ;
- empreinte mémoire : la layer de base distroless tient en quelques MB en RAM, là où Debian ou Ubuntu déchargent plusieurs dizaines de MB de bibliothèques qu'aucun process n'utilise.

L'agrégation de ces gains sur l'ensemble du cycle de vie (chaque rolling update, chaque scale-up HPA, chaque redéploiement complet via `just redeploy`) reste structurelle plutôt que ponctuelle : chaque mécanisme de la plateforme consomme moins de ressources parce que l'image source est plus petite.

4.10.4 Mutualisation du cache CI

Le cache GitHub Actions est scope par composant (cf. §4.3.5 et §4.5.2) : trois caches indépendants pour api, worker et audit-purge. L'option `mode=max` exporte toutes les couches intermédiaires, ce qui maximise les hits sur builds successifs.

L'effet observé sur un rebuild incrémental (modification d'un fichier `.go` côté api seul, déjà cité en §4.3.5) : le job build du composant api retombe autour de 1 minute en cache chaud contre 4 à 5 minutes en cache froid, et les jobs worker et audit-purge restent intacts. Pour un cycle de cinq commits successifs touchant uniquement l'api, l'économie cumulée s'élève à environ 20 minutes de runner GitHub par série, plus la bande passante de pull des modules Go qui n'est plus consommée.

Le coût carbone individuel d'une minute de runner reste modeste, mais la pratique de mutualisation s'inscrit dans le principe directeur de sobriété par défaut (cf. §4.1.4) : chaque rebuild qui n'a pas besoin d'exister est un rebuild qui n'est pas effectué.

4.10.5 Rétentions courtes des données d'observabilité

Les trois bases d'observabilité retiennent leurs données pendant sept jours :

- Prometheus, sept jours (rétention par défaut du chart kube-prometheus-stack, PVC de 10 Gi) ;
- Loki, 168 heures soit sept jours (cf. *kubernetes/applications/monitoring/loki.yaml:36*, PVC de 10 Gi) ;
- Tempo, 168 heures (cf. *kubernetes/applications/monitoring/tempo.yaml:19*, PVC de 10 Gi).

Le choix d'une rétention courte plutôt que la valeur usuelle de 30 ou 90 jours s'appuie sur trois constats. La grande majorité des investigations post-incident porte sur des événements de moins d'une semaine. Au-delà, les enquêtes deviennent rares et la pertinence des traces ou des logs anciens diminue. La rétention courte aligne l'observabilité sur le profil de l'audit log RGPD (cf. §4.6.5 et §4.7.7), ce qui simplifie la cohérence des durées de conservation.

L'économie de stockage est significative : passer à 30 jours imposerait environ 30 Gi additionnels par base sur trois bases, ce qui saturerait l'espace disque disponible en quelques semaines de fonctionnement nominal. Le seuil sept jours rend l'utilisation continue sans intervention manuelle.

Pour les tendances long-terme (analyse mensuelle, comparaison annuelle), l'alternative consisterait à exporter les recording rules Prometheus vers une base remote-write longue durée (Mimir, Cortex, Thanos). Cette extension dépasse le périmètre M1 ; elle reste documentée comme évolution possible en §4.10.6.

4.10.6 Reporting Green IT et limites connues

Reporting actuel. Les dashboards Grafana exposent la consommation CPU et RAM par namespace (cf. §4.8.5, dashboard *Platform overview*), ce qui couvre le KPI minimum du cahier (*docs/cahier-des-charges.pdf* §IX, « consommation CPU/RAM par environnement »). Les métriques Kepler sont accessibles dans Prometheus mais ne disposent pas d'un dashboard direction dédié. Les KPI implicites (joules par requête, joules par job traité) sont calculables en PromQL mais ne sont pas matérialisés dans un panel.

Trois limites bornent la portée du reporting Green IT actuel.

Première limite, la précision Kepler en VM. Le modèle ML pré-entraîné fournit un ordre de grandeur exploitable pour comparer Pods et namespaces entre eux, mais pas une valeur absolue calibrée à un wattmètre. Un reporting direction signé exigerait soit une calibration sur bare-metal avec RAPL accessible, soit une corroboration externe (mesure de consommation au mur de l'hôte Proxmox, attribution par règle de trois).

Deuxième limite, l'absence de comparaison historique au-delà de sept jours. Sans Mimir ou Thanos, les tendances trimestrielles ne sont pas disponibles. Un agrégat annuel ne peut être ni produit ni vérifié.

Troisième limite, l'absence de scope FinOps. Le coût économique des ressources (heure de VM, transferts réseau, stockage) n'est pas calculé. La transposition Joules vers euros vers tonnes CO2

exigerait l'ajout d'un outil dédié (Cloud Carbon Footprint, Kepler-Model-Server pour des modèles avancés) et un mapping client par client en cas de multi-tenancy. L'ensemble relève du M2 (cf. §1.4).

Les conditions techniques d'évolution vers un reporting de production sont les suivantes. Calibrer Kepler sur bare-metal une fois et conserver le modèle calibré comme référence VM. Ajouter un dashboard Grafana dédié au reporting direction (joules par requête, joules par client, tendance hebdomadaire). Activer une base remote-write longue durée pour les tendances annuelles. Intégrer un outil de Cloud Carbon Footprint pour la conversion en équivalents CO2. Ces évolutions s'inscrivent dans la roadmap M2 FinOps plus DevSecOps, en cohérence avec le découpage assumé du périmètre M1.

4.11 KPIs et résultats observés

La présente section consolide les KPIs mesurables sur le snapshot du 24 mai 2026 (commit afe314f sur la branche *main*). Chaque valeur citée est sourcée directement depuis le projet (manifests, workflows, code applicatif) ou observable via les commandes reproductibles documentées. Les valeurs qui ne sont pas figées au snapshot (couverture de tests, uptime cluster, MTTR) sont accompagnées de la commande d'extraction au jour de soumission.

4.11.1 KPIs ancrés au snapshot du 24 mai 2026

Le tableau ci-dessous synthétise les KPIs vérifiables au snapshot. La colonne *source* renvoie au fichier ou à la commande qui produit la valeur, ce qui rend chaque ligne auditable.

KPI	Valeur observée	Cible §2.2.1	Source
Taux de succès CI infrastructure (validate.yml)	environ 96 % (24/25 derniers runs)	au-dessus de 90 %	gh run list -- workflow=validate.yml
Taux de succès CI application (build.yml)	environ 53 % (8/15 derniers runs)	au-dessus de 90 %	gh run list -- workflow=build.yml
Taux de succès CI application sur les 4 derniers runs	100 %	au-dessus de 90 %	idem, limit=4
Durée moyenne CI infrastructure	environ 35 secondes	non spécifié	delta startedAt → updatedAt
Durée moyenne CI application	1 à 5 minutes	non spécifié	idem
Deploy time bout en bout (push Git → pod ready)	3 à 5 minutes	sous 5 minutes	observable via kubectl get pods -w plus historique ArgoCD
Vulnérabilités HIGH/CRITICAL bloquantes en images publiées	0	0	dernier run build.yml réussi
SLO de disponibilité cible	99.5 %	99.5 %	kubernetes/apps/projet-etude-app-

			<i>demo/prometheusrule.yaml:42,55</i>
Burn-rate fast trigger	14.4× sur 5m et 1h pendant 2m	non spécifié	<i>prometheusrule.yaml:43-44</i>
Nombre de recording rules	4	non spécifié	<code>grep -c "^ - record" prometheusrule.yaml</code>
Nombre d'alerting rules	7 (5 plus 2 SLO burn-rate)	non spécifié	<code>grep -c "^ - alert" prometheusrule.yaml</code>
Conso CPU requestée par la plateforme (hors application)	environ 0.6 vCPU	non spécifié	sommation §4.10.2
Conso RAM requestée par la plateforme (hors application)	environ 1.8 GiB	non spécifié	sommation §4.10.2
Conformité kube-score	passé avec 4 tests ignorés et justifiés	passé	<i>.github/workflows/validate.yml</i>

Le dashboard Kubernetes de consommation cluster (kube-prometheus-stack) corrobore visuellement ces valeurs : utilisation CPU et mémoire globales, et ventilation CPU/RAM par namespace.



Dashboard Kubernetes — consommation CPU/RAM du cluster par namespace

Le taux de succès `build.yml` à 53 % nécessite une lecture nuancée. La fenêtre de 15 runs analysée couvre la période de mise au point initiale de deux dépendances : l'action `golangci/golangci-lint-action@v7` (changement de schéma de configuration) et `aquasecurity/trivy-`

action@v0.36.0 (changement d'interface CLI). Sur les quatre derniers runs avant le snapshot, le taux atteint 100 %, ce qui indique la stabilisation post-mise-au-point. La cible §2.2.1 (« au-dessus de 90 % ») reste à valider sur les 30 derniers runs au jour de soumission, par la commande `gh run list --workflow=build.yml --limit 30 --json conclusion | jq`.

La décomposition du deploy time bout en bout, déjà donnée en §4.5.5, se vérifie en suivant :

- pipeline build.yml côté application : 1 à 5 minutes, dominée par le test race et les trois builds matrix ;
- détection du nouveau digest GHCR par argocd-image-updater : intervalle de polling 2 minutes par défaut ;
- write-back Git et re-sync ArgoCD : moins d'une minute ;
- rolling update du Deployment sous contrainte PDB : moins de 30 secondes.

4.11.2 KPIs à actualiser au jour de soumission

Cinq KPIs sont mesurables au runtime mais non figés dans le présent DAT. Ils requièrent une extraction au jour de soumission, idéalement à partir d'un cluster actif. Les commandes sont fournies pour rendre l'opération reproductible.

KPI	Cible §2.2.1	Commande d'extraction
Couverture de tests Go	non spécifiée	<code>cd projet-etude-app-demo && go test -cover ./...</code>
Uptime cluster depuis le 11 mai 2026	non spécifiée	requête Prometheus <code>avg_over_time(up{job="apiserver"}[<period>])</code> sur la fenêtre concernée
MTTR effectif sur incidents observés	non spécifiée	extraction historique Alertmanager plus corrélation Git
Performance applicative sous charge : p95 à 200 RPS soutenus	sous 1 s	kubernetes/loadtest/job.yaml exécute hey 90 s, plus dashboard SLO
Consommation énergétique observée par Pod	non spécifiée	requête Prometheus <code>rate(kepler_container_joules_total[5m])</code> par container

La cible de performance p95 à 200 RPS soutenus s'évalue par exécution du Job de charge (just loadtest) sur le snapshot du 24 mai ou ultérieur, avec lecture parallèle du dashboard SLO. Une exécution unique de référence sera produite avant la vidéo MVP, et l'écart à la cible sera mentionné dans le commentaire de soumission si l'écart dépasse 10 %.

L'uptime cluster sera calculé à partir de la date du premier déploiement complet (11 mai 2026, J5 du calendrier projet, cf. §3.4.1). Au jour de soumission le 19 juin 2026 (J7), la fenêtre d'observation théorique est d'environ 39 jours. La valeur réelle dépend des redéploiements volontaires (just redeploy complet) qui interrompent le cluster pendant quelques minutes.

4.11.3 Synthèse architecture quantifiée

Les éléments quantifiés ci-dessous décrivent le volume du projet livré au snapshot. Ils servent à la fois de mesure de l'effort produit et de base de comparaison pour les évolutions ultérieures.

Infrastructure NixOS : 12 fichiers Nix (1 flake, 8 modules, 3 hosts) et 5 scripts shell Proxmox, environ 1 100 lignes de configuration déclarative et de script Bash.

Recettes opérationnelles : 47 recettes just documentées via l'introspection `just --list`.

Manifests Kubernetes versionnés : environ 38 fichiers YAML applicatifs et plateforme.

ArgoCD Applications gérées : environ 12 (kube-prometheus-stack, Loki, Alloy, Tempo, Vault, External-Secrets, Dex, trivy-operator, Kepler, CloudNative-PG, ApplicationSet apps, plus la root app posée par `k8s-bootstrap.nix`).

Namespaces fonctionnels : 8 (argocd, monitoring, security, external-secrets, trivy-system, kepler, cnpg-system, projet-etude-app-demo).

Application Go : environ 2 277 lignes de code non-test sous `cmd/` et `internal/`, plus 949 lignes de tests réparties sur 7 fichiers `*_test.go`.

Dashboards Grafana custom : 4 (Platform overview, App demo métier, SLO et burn-rate, Worker et queue), tous injectés via ConfigMap label `grafana_dashboard: "1"`.

PrometheusRules : 4 recording rules et 7 alerting rules dans `prometheusrule.yaml`.

NetworkPolicies dédiées : 4 (api, worker, audit-purge, chaos-killer).

Le volume reste cohérent avec un projet d'études M1 conduit par quatre étudiants sur six mois et demi. La densité par membre (environ 13 manifests Kubernetes, 12 recettes just et 250 lignes Go par membre) est compatible avec la disponibilité d'une équipe en alternance, à condition d'avoir tenu la cadence Scrumban d'un sprint hebdomadaire (cf. §3.1.1 et §3.4.3).

4.12 Limites techniques et perspectives

Les limitations assumées au cours du projet ont été documentées au fil du DAT (cf. §2.6 pour la matrice synthétique et §4.7.8 pour les conditions de passage en production). La cartographie ci-dessous regroupe les six axes d'évolution prospectifs identifiés, chacun orienté vers un cap technique précis et tagué par son périmètre. Le développement détaillé de ces axes appartient aux rapports individuels, qui les reprennent selon le périmètre éditorial de chaque membre.

- **Vault, OIDC et rotation des secrets** (périmètre M1, sécurité) : passage de Vault DEV à un mode standalone ou ha avec scellement KMS, restauration OIDC réelle vers Dex (connecteurs GitHub, LDAP ou SAML selon le client), automatisation de la rotation `sops updatekeys` et des tokens Vault. Détail dans le rapport de Nicolas Roulois.
- **Backup BDD et plan de reprise d'activité** (périmètre M1, résilience) : activation de `backup.barmanObjectStore` côté CNPG vers un stockage S3-compatible, WAL archiving continu, drills de restauration trimestriels avec cibles RPO/RTO, procédure PRA versionnée. Limite résiduelle la plus critique. Détail dans les rapports de Tom Patelas et Karim Haddadi.
- **Haute disponibilité et stockage répliqué** (périmètre M1, résilience) : passage de Postgres à instances : 3, control-plane K3s en HA etcd à trois nœuds (`--cluster-init + --server`),

remplacement du local-path par Longhorn ou Rook-Ceph. Détail dans le rapport de Tom Patelas.

- **Mises à jour automatisées des dépendances** (périmètre M1, supply chain) : adoption de Renovate (plutôt que Dependabot) couvrant modules Go, charts Helm, actions GitHub et images de base ; auto-merge conditionnel des patches sécurité après CI verte. Détail dans le rapport de Karim Haddadi.
- **Documentation utilisateur et OpenAPI** (périmètre M1, documentation) : guide utilisateur final via MkDocs sur GitHub Pages, spécification OpenAPI 3 exposée sur /api/openapi.json avec interface Swagger UI, guide d'administration consolidé pour le pôle Exploitation.
- **Multi-tenancy, FinOps et reporting carbone** (périmètre M2, déjà cadré §1.4) : isolation par namespace par client avec ResourceQuota/LimitRange/RBAC scoped, Kubecost ou OpenCost pour le coût économique, Cloud Carbon Footprint pour l'équivalent CO₂ par tenant, dashboard direction agrégeant euros, kWh et gCO₂ par client. L'instrumentation actuelle (Kepler, métriques requests, dashboards de consommation) pose les fondations techniques de cette couche M2 sans la matérialiser.

Glossaire technique

- **ApplicationSet** : ressource ArgoCD qui génère dynamiquement des Applications à partir d'un générateur (Git, list, cluster).
- **ArgoCD** : outil de déploiement continu déclaratif pour Kubernetes, suit le pattern GitOps.
- **CNCF** : Cloud Native Computing Foundation, fondation qui héberge les projets open-source cloud-native (Kubernetes, Prometheus, etc.).
- **CNPG** : CloudNative-PG, opérateur Kubernetes pour PostgreSQL.
- **CRD** : Custom Resource Definition, extension Kubernetes qui définit une nouvelle ressource gérable comme les ressources natives.
- **DAT** : Document d'Architecture Technique.
- **Disko** : outil Nix qui déclare le partitionnement et le formatage des disques de manière déclarative.
- **DSAR** : Data Subject Access Request, demande d'accès aux données personnelles au titre du RGPD.
- **ESO** : External-Secrets Operator, opérateur qui synchronise les secrets d'un coffre-fort externe (Vault, AWS Secrets Manager, etc.) vers les Secrets Kubernetes.
- **Exemplar (Prometheus)** : échantillon associé à un bucket d'histogramme qui transporte un identifiant externe (typiquement un trace_id).
- **Flake (Nix)** : format reproductible et versionné du gestionnaire de paquets Nix, qui pin chaque dépendance.
- **Flannel** : CNI (Container Network Interface) léger utilisé par défaut dans K3s.
- **GHCR** : GitHub Container Registry.
- **GitOps** : méthodologie où Git est la source unique de vérité pour la configuration déclarative de l'infrastructure et des applications.
- **HPA** : Horizontal Pod Autoscaler, ressource Kubernetes qui ajuste le nombre de répliques d'un Deployment à la charge.
- **HTMX** : bibliothèque JavaScript légère qui étend HTML avec des attributs hx-* pour des interactions asynchrones sans framework SPA.
- **K3s** : distribution Kubernetes légère packagée par Rancher (SUSE), conçue pour edge et bare-metal restreint.
- **Kepler** : Kubernetes Efficient Power Level Exporter, DaemonSet qui mesure ou estime la consommation énergétique par container.
- **kexec** : mécanisme du kernel Linux qui charge à chaud un nouveau kernel sans passer par le BIOS/UEFI.
- **Kubeconform** : validateur de manifests Kubernetes contre les schémas officiels et CRDs externes.
- **Kustomize** : outil de patching de manifests Kubernetes intégré à kubectl.
- **LISTEN/NOTIFY** : mécanisme de publish/subscribe natif PostgreSQL.
- **Loki** : système d'agrégation de logs de Grafana Labs, indexé par labels.
- **MTTD** : Mean Time To Detect, durée moyenne entre l'occurrence d'un incident et sa détection.

- **MTTR** : Mean Time To Recovery, durée moyenne entre la détection d'un incident et son rétablissement.
- **NetworkPolicy** : ressource Kubernetes qui définit les règles de trafic réseau entre Pods (ingress/egress).
- **NixOS** : distribution Linux déclarative basée sur le langage et le gestionnaire de paquets Nix.
- **nixos-anywhere** : outil qui déploie une configuration NixOS sur une machine distante via kexec, sans accès console.
- **OIDC** : OpenID Connect, protocole d'authentification basé sur OAuth 2.0.
- **OpenTelemetry (OTel)** : spécification standardisée d'instrumentation pour métriques, logs et traces.
- **OTLP** : OpenTelemetry Protocol, format wire d'exportation des données OTel.
- **PDB** : PodDisruptionBudget, ressource Kubernetes qui garantit un seuil de disponibilité minimum pendant les évictions volontaires.
- **pgx** : driver PostgreSQL natif Go, mature, performant, qui supporte LISTEN/NOTIFY.
- **PRA** : Plan de Reprise d'Activité.
- **Prometheus** : système de monitoring et d'alerting time-series, standard CNCF.
- **PVC** : PersistentVolumeClaim, demande de stockage persistant dans Kubernetes.
- **RAPL** : Running Average Power Limit, compteurs énergétiques exposés par les CPU Intel et AMD modernes.
- **RBAC** : Role-Based Access Control, modèle de contrôle d'accès basé sur les rôles.
- **RGPD** : Règlement Général sur la Protection des Données.
- **RollingUpdate** : stratégie de déploiement Kubernetes qui remplace progressivement les anciens Pods par les nouveaux.
- **Scrumban** : méthodologie hybride entre Scrum (sprints, cérémonies) et Kanban (flux continu, WIP limits).
- **SLO** : Service Level Objective, cible quantifiée de qualité de service.
- **sops** : Secret OPERationS, outil Mozilla de chiffrement de fichiers de configuration avec recipients multiples.
- **sops-nix** : intégration NixOS de sops, qui déchiffre les secrets au boot via la clé hôte.
- **Tailscale** : VPN mesh basé sur WireGuard, avec contrôle centralisé.
- **Tailscale serve** : fonctionnalité Tailscale qui termine TLS et expose un service HTTPS sur le tailnet privé.
- **Tempo** : système de traces distribuées de Grafana Labs, compatible OTLP.
- **testcontainers-go** : bibliothèque Go qui démarre des conteneurs Docker éphémères pour les tests d'intégration.
- **Traefik** : reverse proxy moderne, intégré par défaut dans K3s comme ingress controller.
- **Trivy** : scanner de vulnérabilités d'images conteneurs, d'IaC et de dépendances.
- **Vault** : gestionnaire de secrets HashiCorp, centralisé et auditable.